

머신러닝 분류

- 평가(Evaluation)

- > 머신러닝은 데이터 가공/변환(전처리), 모델 학습/예측, 평가의 프로세스로 구성된다.
- > 성능 평가 지표는 일반적으로 모델이 분류냐 회귀냐에 따라 여러 종류로 나뉜다.
- > 분류의 평가방법은 실제 결과 데이터와 예측 결과 데이터가 얼마나 정확하고 오류가 적게 발생하는가에 기반한다.
- > 회귀의 평가방법은 실제값과 예측값의 오차 평균값에 기반한다.

• 평가(Evaluation)

> 분류 모델의 성능 평가 지표의 종류

- 정확도(Accuracy)
- 오차행렬(Confusion Matrix)
- 정밀도(Precision)
- 재현율(Recall)
- F1 스코어
- ROC AUC

> 회귀 모델의 성능 평가 지표의 종류

- MAE(Mean Absolute Error)
- MSE(Mean Squared Error)
- RMSE(Root Mean Squared Error)
- R^2

- 정확도

> 정확도는 실제 데이터에서 예측 데이터가 얼마나 같은지를 판단하는 지표

$$\text{정확도}(Accuracy) = \frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}}$$

> 가장 직관적으로 모델 예측 성능을 나타내는 평가 지표

> 그러나 이진 분류의 경우 데이터의 구성에 따라 성능을 왜곡할 수 있기에 **정확도 수치 하나만 가지고 모델 성능을 평가하지 않는다.**

- 정확도

- > 성능 왜곡 실습

```
from sklearn.base import BaseEstimator
# 남성이면 사망, 여성이면 생존으로 예측하는 모델
class MyDummyClassifier(BaseEstimator):
    def fit(self, X, y = None):
        pass
    def predict(self, X):
        pred = np.zeros((X.shape[0], 1))
        for i in range(X.shape[0]):
            if X['Sex'].iloc[i] == 'male':
                pred[i] = 0
            else:
                pred[i] = 1
        return pred
```

- 정확도

- > 성능 왜곡 실습

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

titanic_df = pd.read_csv('data/titanic.csv')

y_titanic_df = titanic_df['Survived']
X_titanic_df = titanic_df.drop('Survived', axis = 1)

X_train, X_test, y_train, y_test = train_test_split(X_titanic_df, y_titanic_df,
                                                    test_size = 0.2, random_state = 0)
```

- 정확도

- > 성능 왜곡 실습

```
# 생성한 모델을 사용하여 학습/예측/평가
myclf = MyDummyClassifier()
myclf.fit(X_train, y_train)
mypredictions = myclf.predict(X_test)
print('정확도 : ', accuracy_score(y_test, mypredictions))
```

Out : 정확도 : 0.7877094972067039

- 정확도

- > 불균형 데이터 예측

```
# 학습시 아무런 행동을 하지않고 0으로 예측하는 모델 생성
class MyFakeClassifier(BaseEstimator):
    def fit(self,X,y):
        pass
    def predict(self,X):
        return np.zeros( (len(X), 1))
```

- 정확도

- > 불균형 데이터 예측

```
from sklearn.datasets import load_digits
```

```
digits = load_digits()
```

```
print(digits.data)
```

```
print(digits.target)
```

```
# 번호가 7이면 True==1, 그 외는 False==0
```

```
y = (digits.target == 7).astype(int)
```

```
X_train, X_test, y_train, y_test = train_test_split( digits.data, y, random_state=11)
```

```
Out : [[ 0.  0.  5. ...  0.  0.  0.]  
       [ 0.  0.  0. ... 10.  0.  0.]  
       [ 0.  0.  0. ... 16.  9.  0.]  
       ...  
       [ 0.  0.  1. ...  6.  0.  0.]  
       [ 0.  0.  2. ... 12.  0.  0.]  
       [ 0.  0. 10. ... 12.  1.  0.]]  
[0 1 2 ... 8 9 8]
```

- 정확도

- > 불균형 데이터 예측

```
# 불균형한 레이블 데이터 분포도 확인
print('레이블 테스트 세트 크기 :', y_test.shape)
print('테스트 세트 레이블 0 과 1의 분포도')
print(pd.Series(y_test).value_counts())

# 생성한 모델로 학습/예측/평가
fakeclf = MyFakeClassifier()
fakeclf.fit(X_train , y_train)
fakepred = fakeclf.predict(X_test)
print(f'정확도:{accuracy_score(y_test , fakepred):.3f}')
```

- 정확도

- > 불균형 데이터 예측

Out : 레이블 테스트 세트 크기 : (450,)
테스트 세트 레이블 0 과 1의 분포도
0 405
1 45
dtype: int64
모든 예측을 0으로 하여도 정확도는 :0.900

결과값이 전부 0임에도 불구하고 테스트 데이터의 90%가 0을
가지고 있어 정확도는 90%에 해당한다.

이러한 한계점을 극복하기 위해서 여러 가지 분류 지표와 함께
사용한다.

• 오차 행렬

012

- > 학습된 분류 모델이 예측한 분류와 실제 데이터의 분류 범주를 교차 표로 나타낸 것이다.
- > 4분면 행렬에서 왼쪽, 오른쪽을 예측 값 기준으로 Negative, Positive로 분류하고, 위, 아래를 실제 값 기준으로 Negative, Positive로 분류한다.

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	1종 오류 FP (False Positive)
	Positive(1)	FN (False Negative) 2종 오류	TP (True Positive)

- 오차 행렬

> confusion_matrix()

```
# 앞에서 만든 MyFakeClassifier의 예측 결과의 오차행렬
from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, fakepred)
```

Out : array([[405, 0],
 [45, 0]], dtype=int64)

> TP, TN, FP, FN 값을 조합하여 주요 지표인 정확도, 정밀도, 재현율 값을 알 수 있다.

$$\text{정확도} = \frac{\text{예측 결과와 실제 값이 동일한 건수}}{\text{전체 데이터 수}} = \frac{(TN + TP)}{(TN + FP + FN + TP)}$$

• 정밀도와 재현율

- > 정밀도와 재현율은 Positive 데이터 세트의 예측 성능에 초점을 맞춘 평가 지표이다.
- > 정밀도와 재현율은 다음과 같은 공식으로 계산한다.

$$\text{정밀도} = \frac{TP}{(FP + TP)}$$

$$\text{재현율} = \frac{TP}{(FN + TP)}$$

- > 정밀도는 예측 값이 Positive일 때, TP 비율
 - 양성으로 예측한 가운데 맞춘 양성의 수
 - 얼마나 정확하게 예측하는지에 대한 지표
- > 재현율은 실제 값이 Positive일 때, TP 비율
 - 실제 양성 중 양성으로 판단한 비율
 - 실제 양성에 대해서 얼마나 정확하게 양성이라고 예측하는지에 대한 지표

• 정밀도와 재현율

- > 정밀도와 재현율 중 중요시 해야하는 점
- > 정밀도는 실제 Negative 데이터를 Positive로 잘못 판단하는 일이 중요한 경우
 - 정밀도가 낮다: 거짓인데 참이라고 한 것이 많다.
 - 스팸 메일이면 참, 스팸 메일이 아니면 거짓
 - 스팸 메일이 아닌데 스팸 메일로 판단해서 차단해버리면 중요한 메일을 받지 못할 수 있다.
- > 재현율은 실제 Positive 데이터를 Negative로 잘못 판단하는 일이 중요한 경우
 - 재현율이 낮다: 참인데 거짓이라고 한 것이 많다.
 - 암이 검출되면 참, 검출되지 않으면 거짓
 - 실제로 암에 걸렸는데 걸리지 않았다고 판단하는 경우가 가장 위험

- 정밀도와 재현율

> precision_score, recall_score

```
from sklearn.metrics import precision_score, recall_score
```

```
precision = precision_score(y_test , fakepred)
```

```
recall = recall_score(y_test , fakepred)
```

```
print(f'정밀도: {precision:.4f}, 재현율: {recall:.4f}')
```

Out : 정밀도: 0.0000, 재현율: 0.0000

- 정밀도와 재현율

> precision_score, recall_score

```
# 타이타닉 데이터
titanic_df = pd.read_csv('data/titanic.csv')

titanic_df['Age'].fillna(titanic_df['Age'].mean(), inplace=True)
titanic_df['Cabin'].fillna('N', inplace=True)
titanic_df['Embarked'].fillna('N', inplace=True)
titanic_df['Cabin'] = titanic_df['Cabin'].str[:1]
y_titanic_df = titanic_df['Survived']
X_titanic_df = titanic_df.drop(['PassengerId', 'Survived', 'Name', 'Ticket'], axis = 1)
X_titanic_df = pd.get_dummies(X_titanic_df)
```

- 정밀도와 재현율

018

> precision_score, recall_score

```
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(X_titanic_df, y_titanic_df,
                                                    test_size = 0.2, random_state=0)

lr_clf = LogisticRegression(solver='liblinear')
lr_clf.fit(X_train , y_train)
pred = lr_clf.predict(X_test)
```

Out : 오차 행렬
[[92 18]
[20 49]]
정확도: 0.7877, 정밀도: 0.7313, 재현율: 0.7101

- 정밀도와 재현율

019

> precision_score, recall_score

```
confusion = confusion_matrix( y_test, pred)
accuracy = accuracy_score(y_test , pred)
precision = precision_score(y_test , pred)
recall = recall_score(y_test , pred)
print('오차 행렬')
print(confusion)
print(f'정확도: {accuracy:.4f}, 정밀도: {precision:.4f}, 재현율: {recall:.4f}')
```

Out : 오차 행렬
[[94 16]
 [18 51]]
정확도: 0.8101, 정밀도: 0.7612, 재현율: 0.7391

• 정밀도와 재현율

> 정밀도/재현율 트레이드오프

- 업무의 특성상 정밀도 또는 재현율이 특별히 강조돼야 할 경우 임곗값(Threshold)을 조정하여 수치를 높일 수 있다.
- 하지만 정밀도와 재현율은 상호 보완적인 평가 지표이기에 한쪽을 높이면 다른 하나의 수치는 떨어지기 쉽다.

> 개별 레이블 별로 결정 확률을 구하고 임곗값을 변경하여 예측 값의 결과를 바꿀 수 있다.

- 정밀도와 재현율

021

> predict_proba

```
pred_proba = lr_clf.predict_proba(X_test)
pred = lr_clf.predict(X_test)
print('pred_proba()결과 Shape :', pred_proba.shape)
print('pred_proba array에서 앞 10개만 샘플로 추출 \n:', pred_proba[:10])
```

Out : pred_proba()결과 Shape : (179, 2)
pred_proba array에서 앞 10개만 샘플로 추출
: [[0.85768298 0.14231702]
[0.89408997 0.10591003]
[0.90769805 0.09230195]
[0.06699819 0.93300181]
[0.3265497 0.6734503]
[0.66873468 0.33126532]
[0.12439344 0.87560656]
[0.09627514 0.90372486]
[0.49693088 0.50306912]
[0.3153 0.6847]]

- 정밀도와 재현율

- > 임계값 조정하기

```
from sklearn.preprocessing import Binarizer
```

```
X = [[ 1, -1,  2],  
      [ 2,  0,  0],  
      [ 0,  1.1, 1.2]]
```

```
binarizer = Binarizer(threshold= 1.1)
```

```
print(binarizer.fit_transform(X))
```

```
Out : [[0. 0. 1.]  
        [1. 0. 0.]  
        [0. 0. 1.]]
```

- 정밀도와 재현율

- > 임계값 조정하기

```
custom_threshold = 0.4
pred_proba_1 = pred_proba[:,1].reshape(-1,1)
binarizer = Binarizer(threshold = custom_threshold ).fit(pred_proba_1)
custom_predict = binarizer.transform(pred_proba_1)

confusion = confusion_matrix( y_test, custom_predict)
accuracy = accuracy_score(y_test , custom_predict)
precision = precision_score(y_test , custom_predict)
recall = recall_score(y_test , custom_predict)
print('오차 행렬')
print(confusion)
print(f'정확도: {accuracy:.4f}, 정밀도: {precision:.4f}, 재현율: {recall:.4f}')
```

- 정밀도와 재현율

- > 임계값 조정하기

Out : 오차 행렬
[[88 22]
[15 54]]
정확도: 0.7933, 정밀도: 0.7105, 재현율: 0.7826

- 정밀도와 재현율

- > 여러 개의 임계값 조정하기

```
thresholds = [0.4, 0.45, 0.50, 0.55, 0.60]

def get_eval_by_threshold(y_test ,pred_proba_c1,thresholds):
    for custom_threshold in thresholds:
        binarizer = Binarizer(threshold = custom_threshold).fit(pred_proba_c1)
        custom_predict = binarizer.transform(pred_proba_c1)
        print('임계값:',custom_threshold)
        confusion = confusion_matrix( y_test, custom_predict)
        accuracy = accuracy_score(y_test , custom_predict)
        precision = precision_score(y_test , custom_predict)
        recall = recall_score(y_test , custom_predict)
        print('오차 행렬')
        print(confusion)
        print(f'정확도: {accuracy:.4f}, 정밀도: {precision:.4f}, 재현율: {recall:.4f}')
```

• 정밀도와 재현율

026

> 여러 개의 임계값 조정하기

```
get_eval_by_threshold(y_test ,pred_proba[:,1].reshape(-1,1), thresholds )
```

Out :

임계값: 0.4
오차 행렬
[[88 22]
 [15 54]]
정확도: 0.7933, 정밀도: 0.7105, 재현율: 0.7826

임계값: 0.45
오차 행렬
[[92 18]
 [16 53]]
정확도: 0.8101, 정밀도: 0.7465, 재현율: 0.7681

임계값: 0.5
오차 행렬
[[94 16]
 [18 51]]
정확도: 0.8101, 정밀도: 0.7612, 재현율: 0.7391

임계값: 0.55
오차 행렬
[[99 11]
 [21 48]]
정확도: 0.8212, 정밀도: 0.8136, 재현율: 0.6957

임계값: 0.6
오차 행렬
[[100 10]
 [24 45]]
정확도: 0.8101, 정밀도: 0.8182, 재현율: 0.6522

- 정밀도와 재현율

> precision_recall_curve

```
from sklearn.metrics import precision_recall_curve

pred_proba_class1 = lr_clf.predict_proba(X_test)[:, 1]
precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_class1 )
print('반환된 분류 결정 임계값 배열의 Shape:', thresholds.shape)
print('반환된 precisions 배열의 Shape:', precisions.shape)
print('반환된 recalls 배열의 Shape:', recalls.shape)
print("thresholds 5 sample:", thresholds[:5])
print("precisions 5 sample:", precisions[:5])
print("recalls 5 sample:", recalls[:5])
```

- 정밀도와 재현율

> precision_recall_curve

Out : 반환된 분류 결정 임계값 배열의 Shape: (172,)
반환된 precisions 배열의 Shape: (173,)
반환된 recalls 배열의 Shape: (173,)
thresholds 5 sample: [0.0363674 0.04863194 0.04880486 0.04888411 0.05267756]
precisions 5 sample: [0.38547486 0.38764045 0.38983051 0.39204545 0.39428571]
recalls 5 sample: [1. 1. 1. 1. 1.]

• 정밀도와 재현율

> precision_recall_curve를 사용한 정밀도, 재현율 곡선

```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

def precision_recall_curve_plot(y_test , pred_proba_c1):
    precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_c1)
    plt.figure(figsize=(8,6))
    threshold_boundary = thresholds.shape[0]
    plt.plot(thresholds, precisions[0:threshold_boundary], '--', label='precision')
    plt.plot(thresholds, recalls[0:threshold_boundary], label='recall')
    start, end = plt.xlim()
    plt.xticks(np.round(np.arange(start, end, 0.1),2))
    plt.xlabel('Threshold value')
    plt.ylabel('Precision and Recall value')
    plt.legend()
    plt.grid()
    plt.show()
```

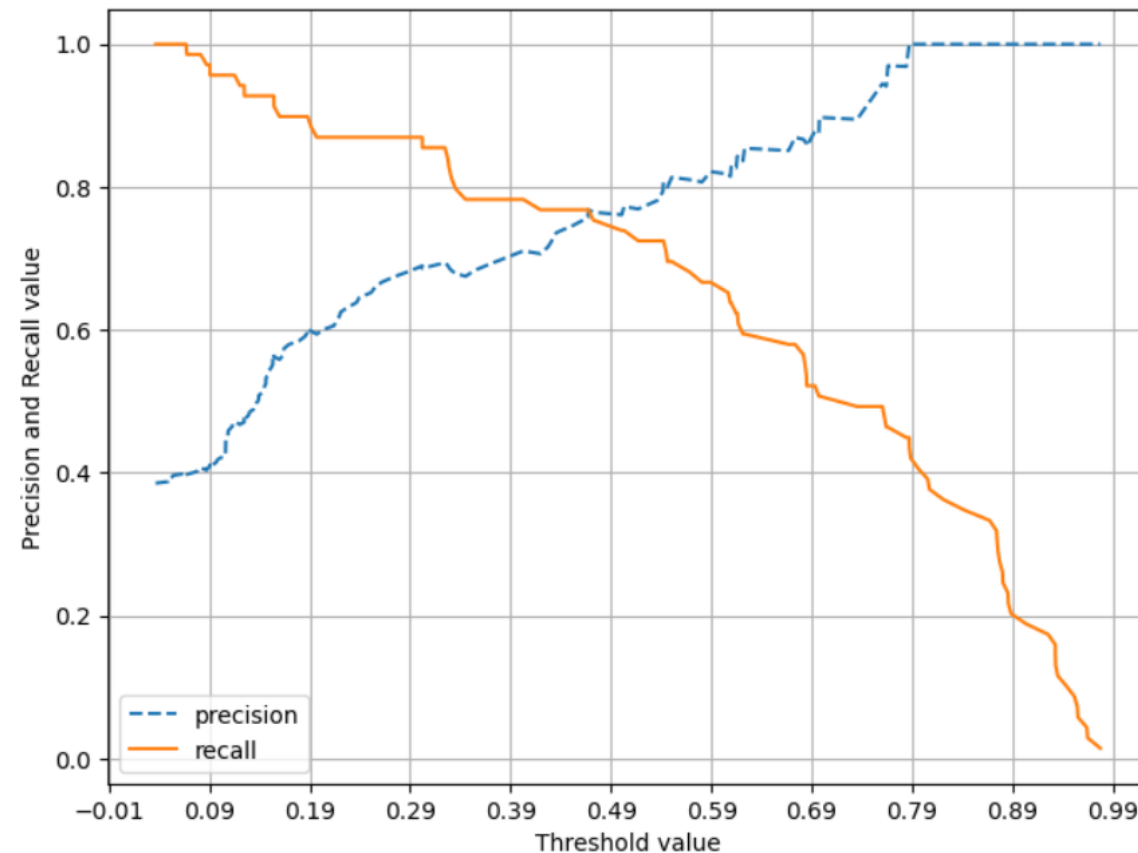
- 정밀도와 재현율

030

> precision_recall_curve를 사용한 정밀도, 재현율 곡선

```
precision_recall_curve_plot(y_test, lr_clf.predict_proba(X_test)[:, 1] )
```

Out :



- F1 스코어

> F1 스코어는 정밀도와 재현율을 결합한 지표이다.

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

> 정밀도, 재현율 어느쪽으로도 치우치지 않을 때, 상대적으로 높은 값을 가진다.

```
from sklearn.metrics import f1_score  
f1 = f1_score(y_test, pred)  
print(f'F1 스코어 : {f1:.4f}')
```

Out : F1 스코어 : 0.7206

- F1 스코어

- > 임계값 조정하기

```
thresholds = [0.4 , 0.45 , 0.50 , 0.55 , 0.60]
pred_proba = lr_clf.predict_proba(X_test)

for custom_threshold in thresholds:
    binarizer = Binarizer(threshold = custom_threshold).fit(pred_proba[:,1].reshape(-1,1))
    custom_predict = binarizer.transform(pred_proba[:,1].reshape(-1,1))
    print('임계값:', custom_threshold)
    accuracy = accuracy_score(y_test , custom_predict)
    precision = precision_score(y_test , custom_predict)
    recall = recall_score(y_test , custom_predict)
    f1 = f1_score(y_test, custom_predict)
    print(f'정확도: {accuracy:.4f}, 정밀도: {precision:.4f}, 재현율: {recall:.4f}, F1: {f1:.4f}')
```

- F1 스코어

- > 임계값 조정하기

Out : 임계값: 0.4
정확도: 0.7933, 정밀도: 0.7105, 재현율: 0.7826, F1: 0.7448
임계값: 0.45
정확도: 0.8101, 정밀도: 0.7465, 재현율: 0.7681, F1: 0.7571
임계값: 0.5
정확도: 0.8101, 정밀도: 0.7612, 재현율: 0.7391, F1: 0.7500
임계값: 0.55
정확도: 0.8212, 정밀도: 0.8136, 재현율: 0.6957, F1: 0.7500
임계값: 0.6
정확도: 0.8101, 정밀도: 0.8182, 재현율: 0.6522, F1: 0.7258

• ROC 곡선과 AUC

- > ROC 곡선과 이에 기반한 AUC 스코어는 이진 분류의 예측 성능 측정에서 중요하게 사용되는 지표이다.
- > ROC 곡선(Receiver Operating Characteristic Curve)은 수신자 판단 곡선으로 불린다.
- > 임계값에 따라 FPR(False Positive Rate)이 변할 때, TPR(True Positive Rate)이 어떻게 변하는지를 나타내는 곡선이다.
- > 임계값을 바꿔도 전반적인 성능이 좋고 나쁘고를 비교하고 싶을 때 사용한다.

$$TPR = \frac{TP}{(FN + TP)} = \text{재현율}$$

$$FPR = \frac{FP}{(FP + TN)} = 1 - \text{TNR} = 1 - \frac{TN}{(FP + TN)}$$

• ROC 곡선과 AUC

035

> roc_curve

```
from sklearn.metrics import roc_curve

pred_proba_class1 = lr_clf.predict_proba(X_test)[: , 1]
fprs , tprs , thresholds = roc_curve(y_test, pred_proba_class1)
thr_index = np.arange(1, thresholds.shape[0], 5)
print('샘플 추출을 위한 임계값 배열의 index:', thr_index)
print('샘플 index로 추출한 임계값: ', np.round(thresholds[thr_index], 2))
print('샘플 임계값별 FPR: ', np.round(fprs[thr_index], 3))
print('샘플 임계값별 TPR: ', np.round(tprs[thr_index], 3))
```

Out : 샘플 추출을 위한 임계값 배열의 index: [1 6 11 16 21 26 31 36 41 46 51 56]
샘플 index로 추출한 임계값: [0.96 0.71 0.64 0.59 0.53 0.43 0.36 0.22 0.16 0.14 0.13 0.08]
샘플 임계값별 FPR: [0. 0.027 0.064 0.091 0.136 0.2 0.227 0.391 0.509 0.618 0.791 0.936]
샘플 임계값별 TPR: [0.014 0.507 0.58 0.652 0.681 0.725 0.812 0.884 0.913 0.942 0.957 0.986]

- ROC 곡선과 AUC

> roc_curve

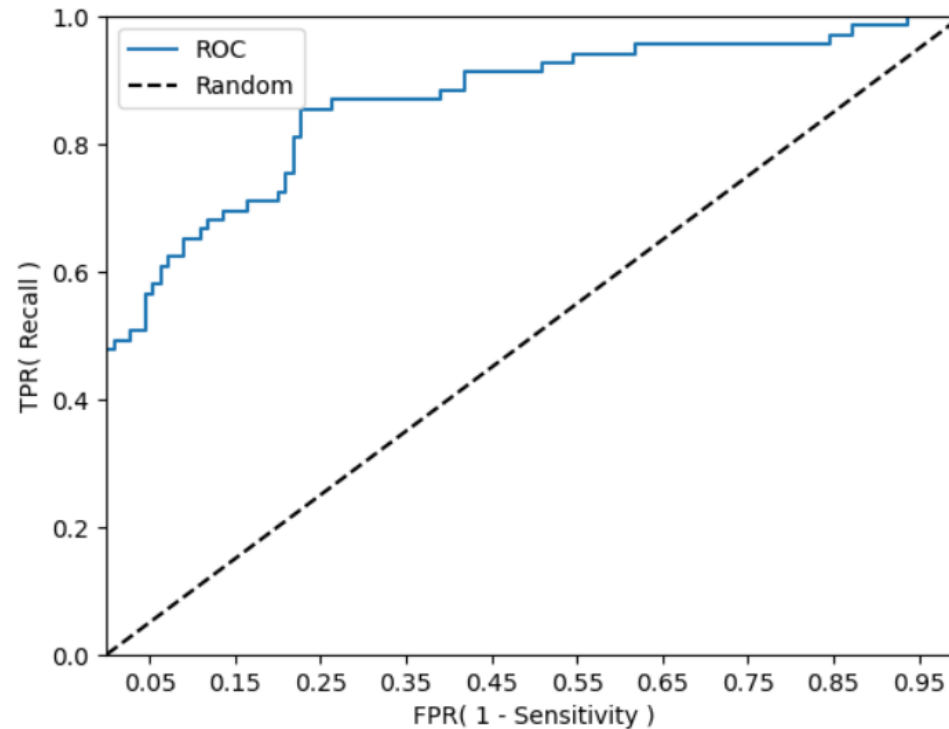
```
def roc_curve_plot(y_test , pred_proba_c1):  
    fprs , tprs , thresholds = roc_curve(y_test ,pred_proba_c1)  
    plt.plot(fprs , tprs, label='ROC')  
    plt.plot([0, 1], [0, 1], 'k--', label='Random')  
    start, end = plt.xlim()  
    plt.xticks(np.round(np.arange(start, end, 0.1),2))  
    plt.xlim(0,1); plt.ylim(0,1)  
    plt.xlabel('FPR( 1 - Specificity )')  
    plt.ylabel('TPR( Recall )')  
    plt.legend()  
    plt.show()
```

- ROC 곡선과 AUC

> roc_curve

```
roc_curve_plot(y_test, lr_clf.predict_proba(X_test)[:, 1] )
```

Out :



- ROC 곡선과 AUC

> roc_curve

```
from sklearn.metrics import roc_auc_score
pred_proba = lr_clf.predict_proba(X_test)[:, 1]
roc_score = roc_auc_score(y_test, pred_proba)
print(f'ROC AUC 값: {roc_score:.4f}')
```

Out : ROC AUC 값: 0.8702

• 피마 인디언 당뇨병 예측

> 피쳐 구성

- Pregnancies : 임신 횟수
- Glucose : 포도당 부하 검사 수치
- BloodPressure : 혈압
- SkinThickness : 팔 삼두근 뒤쪽의 피하지방 측정값
- Insulin : 혈청 인슐린
- BMI : 체질량 지수
- DiabetesPedigreeFunction : 당뇨 내력 가중치 값
- Age : 나이
- Outcome : 클래스 결정 값(0 또는 1)

• 피마 인디언 당뇨병 예측

> 필요 라이브러리 호출

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score
from sklearn.metrics import f1_score, confusion_matrix, precision_recall_curve, roc_curve
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

• 피마 인디언 당뇨병 예측

041

> 데이터 불러오기

```
diabetes_data = pd.read_csv('data/diabetes.csv')
print(diabetes_data['Outcome'].value_counts())
diabetes_data.head(3)
```

Out :

```
0    500
1    268
Name: Outcome, dtype: int64
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1

• 피마 인디언 당뇨병 예측

042

> 데이터 살펴보기

```
diabetes_data.info()
```

```
Out: <class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Pregnancies                          768 non-null    int64
1   Glucose                              768 non-null    int64
2   BloodPressure                        768 non-null    int64
3   SkinThickness                       768 non-null    int64
4   Insulin                              768 non-null    int64
5   BMI                                  768 non-null    float64
6   DiabetesPedigreeFunction             768 non-null    float64
7   Age                                  768 non-null    int64
8   Outcome                              768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

- 피마 인디언 당뇨병 예측

> 데이터 학습/예측/평가

```
X = diabetes_data.iloc[:, :-1]
y = diabetes_data.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                                    random_state = 156, stratify=y)

lr_clf = LogisticRegression(solver='liblinear')
lr_clf.fit(X_train , y_train)
pred = lr_clf.predict(X_test)
pred_proba = lr_clf.predict_proba(X_test)[:, 1]
```

Out :

• 피마 인디언 당뇨병 예측

> 데이터 학습/예측/평가

```
accuracy = accuracy_score(y_test , pred)
confusion = confusion_matrix(y_test, pred)
precision = precision_score(y_test , pred)
recall = recall_score(y_test , pred)
f1 = f1_score(y_test, pred)
roc = roc_auc_score(y_test, pred_proba)
print('오차 행렬')
print(confusion)
print(f'정확도: {accuracy:.4f}, 정밀도: {precision:.4f}, 재현율: {recall:.4f}, F1: {f1:.4f},
ROC_AUC: {roc:.4f}')
```

Out : 오차 행렬
[[87 13]
 [22 32]]

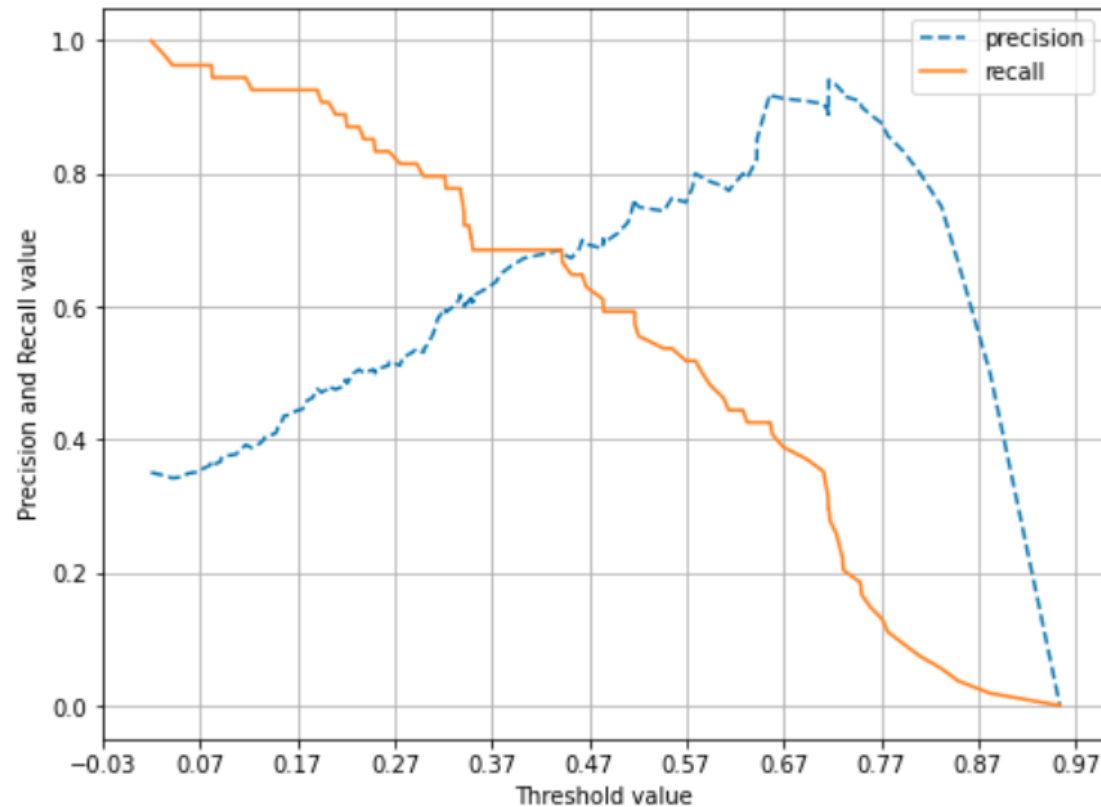
정확도: 0.7727, 정밀도: 0.7111, 재현율: 0.5926, F1: 0.6465, ROC_AUC: 0.8083

- 피마 인디언 당뇨병 예측

> 데이터 학습/예측/평가

```
precision_recall_curve_plot(y_test, pred_proba)
```

Out :



• 피마 인디언 당뇨병 예측

> 피쳐 다시 살피기

```
diabetes_data.describe()
```

Out :

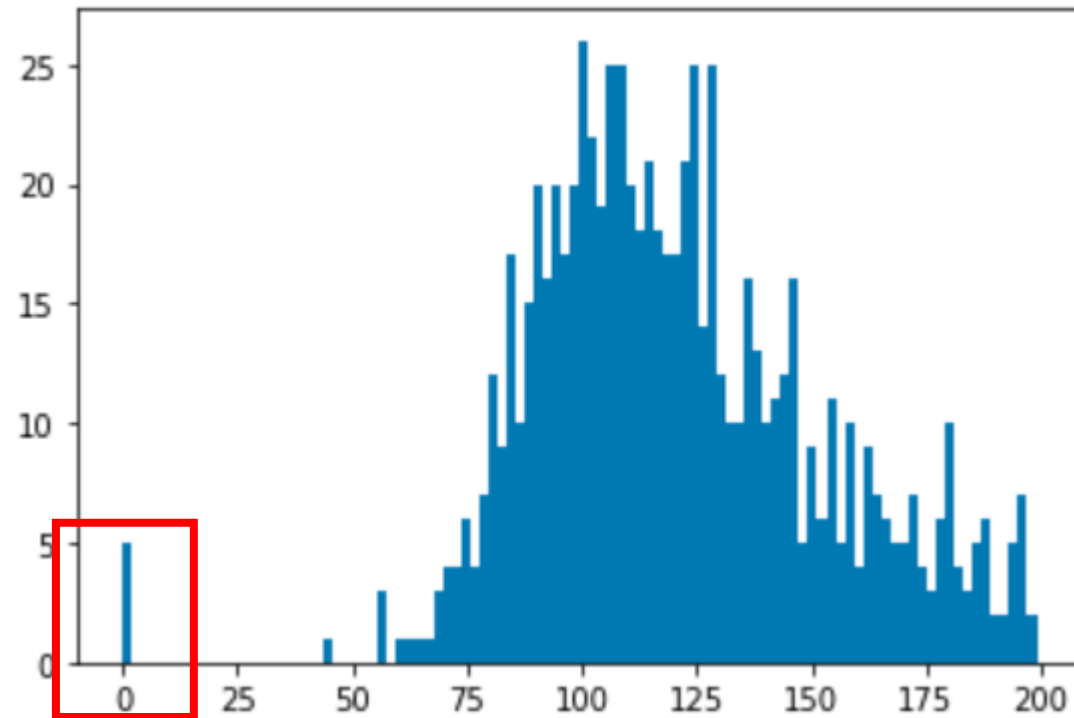
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

- 피마 인디언 당뇨병 예측

> 피쳐 다시 살피기

```
plt.hist(diabetes_data['Glucose'], bins = 100)
```

Out :



• 피마 인디언 당뇨병 예측

> 피쳐 다시 살피기

```
zero_features = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin','BMI']

total_count = diabetes_data['Glucose'].count()
for feature in zero_features:
    zero_count = diabetes_data[ diabetes_data [feature] == 0][feature].count()
    print(f'{feature} 0 건수는 {zero_count}, 퍼센트는 {100*zero_count/total_count:.2f} %')
```

Out : Glucose 0 건수는 5, 퍼센트는 0.65 %
BloodPressure 0 건수는 35, 퍼센트는 4.56 %
SkinThickness 0 건수는 227, 퍼센트는 29.56 %
Insulin 0 건수는 374, 퍼센트는 48.70 %
BMI 0 건수는 11, 퍼센트는 1.43 %

• 피마 인디언 당뇨병 예측

> 피쳐 다시 살피기

```
mean_zero_features = diabetes_data[zero_features].mean()
diabetes_data[zero_features]=diabetes_data[zero_features].replace(0, mean_zero_features)
diabetes_data.describe()
```

Out :

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.681605	72.254807	26.606479	118.660163	32.450805	0.471876	33.240885	0.348958
std	3.369578	30.436016	12.115932	9.631241	93.080358	6.875374	0.331329	11.760232	0.476951
min	0.000000	44.000000	24.000000	7.000000	14.000000	18.200000	0.078000	21.000000	0.000000
25%	1.000000	99.750000	64.000000	20.536458	79.799479	27.500000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	79.799479	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

• 피마 인디언 당뇨병 예측

> 다시 학습/예측/평가

```
X = diabetes_data.iloc[:, :-1]
y = diabetes_data.iloc[:, -1]
scaler = StandardScaler( )
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size = 0.2,
                                                    random_state = 156, stratify=y)

lr_clf = LogisticRegression(solver='liblinear')
lr_clf.fit(X_train , y_train)
pred = lr_clf.predict(X_test)
pred_proba = lr_clf.predict_proba(X_test)[:, 1]
```

• 피마 인디언 당뇨병 예측

> 다시 학습/예측/평가

```
accuracy = accuracy_score(y_test , pred)
confusion = confusion_matrix(y_test, pred)
precision = precision_score(y_test , pred)
recall = recall_score(y_test , pred)
f1 = f1_score(y_test, pred)
roc = roc_auc_score(y_test, pred_proba)
print('오차 행렬')
print(confusion)
print(f'정확도: {accuracy:.4f}, 정밀도: {precision:.4f}, 재현율: {recall:.4f}, F1: {f1:.4f},
ROC_AUC: {roc:.4f}')
```

Out : 오차 행렬
[[90 10]
 [21 33]]
정확도: 0.7987, 정밀도: 0.7674, 재현율: 0.6111, F1: 0.6804, ROC_AUC: 0.8059

• 피마 인디언 당뇨병 예측

> 임계값 변화시키며 성능 평가

```
thresholds = [0.41 ,0.44,0.47, 0.5 , 0.53 ,0.56, 0.59]
```

```
for custom_threshold in thresholds:
```

```
    binarizer = Binarizer(threshold = custom_threshold).fit(pred_proba.reshape(-1,1))
```

```
    custom_predict = binarizer.transform(pred_proba.reshape(-1,1))
```

```
    print('임계값:',custom_threshold)
```

```
    accuracy = accuracy_score(y_test , custom_predict)
```

```
    precision = precision_score(y_test , custom_predict)
```

```
    recall = recall_score(y_test , custom_predict)
```

```
    f1 = f1_score(y_test, custom_predict)
```

```
    print(f'정확도: {accuracy:.4f}, 정밀도: {precision:.4f}, 재현율: {recall:.4f}, F1: {f1:.4f}')
```

• 피마 인디언 당뇨병 예측

> 임계값 변화시키며 성능 평가

Out :

임계값: 0.41
정확도: 0.7532, 정밀도: 0.6481, 재현율: 0.6481, F1: 0.6481
임계값: 0.44
정확도: 0.7597, 정밀도: 0.6667, 재현율: 0.6296, F1: 0.6476
임계값: 0.47
정확도: 0.7857, 정밀도: 0.7234, 재현율: 0.6296, F1: 0.6733
임계값: 0.5
정확도: 0.7987, 정밀도: 0.7674, 재현율: 0.6111, F1: 0.6804
임계값: 0.53
정확도: 0.7987, 정밀도: 0.7805, 재현율: 0.5926, F1: 0.6737
임계값: 0.56
정확도: 0.7922, 정밀도: 0.7750, 재현율: 0.5741, F1: 0.6596
임계값: 0.59
정확도: 0.7792, 정밀도: 0.7632, 재현율: 0.5370, F1: 0.6304

• 분류(Classification)

- > 머신러닝 지도학습의 한 유형으로 레이블 값을 예측한다.
- > 다양한 머신러닝 알고리즘으로 구현할 수 있다.
 - 데이터 균일도에 따른 규칙 기반의 결정 트리(Decision Tree)
 - 서로 다른 머신러닝 알고리즘을 결합한 앙상블(Ensemble)
 - 독립변수와 종속변수의 선형 관계성에 기반한 로지스틱 회귀(Logistic Regression)
 - 개별 클래스 간의 최대 분류 마진을 효과적으로 찾아주는 서포트 벡터 머신(Support Vector Machine)
 - 거리가 가까운 데이터를 참조하는 최근접 이웃(K-Nearest Neighbor)
 - 조건부 확률을 계산하는 방법을 적용한 나이브 베이즈(Naive Bayes)

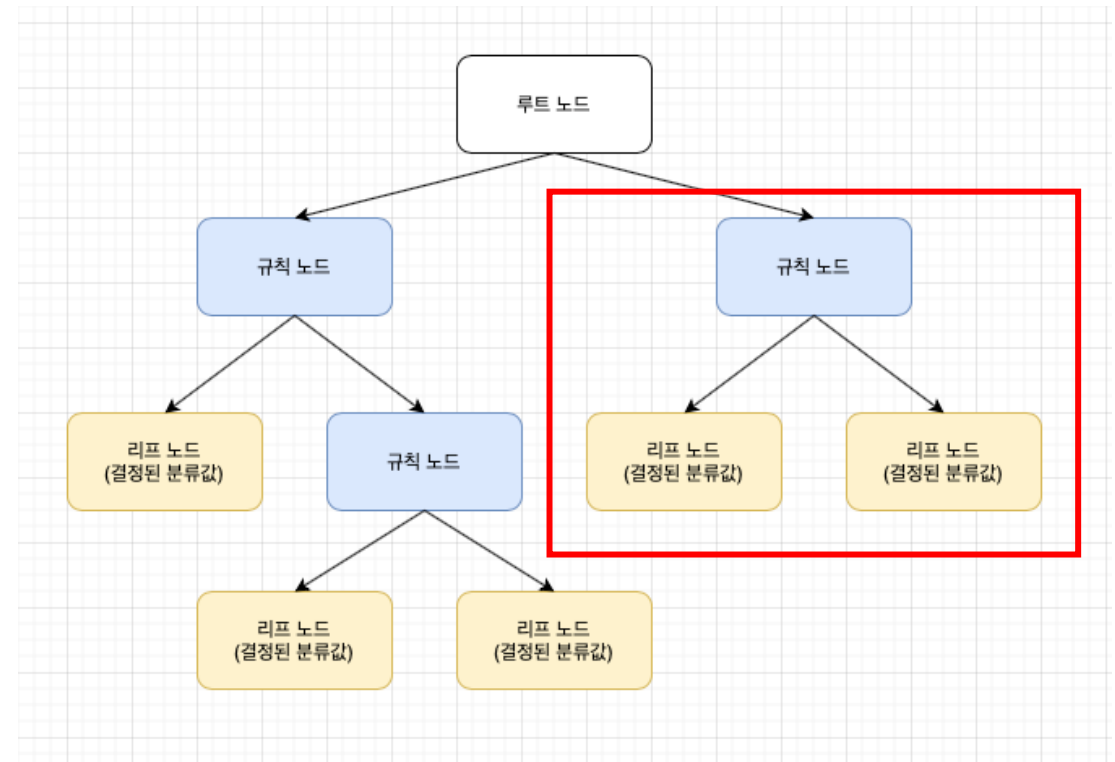
• 결정 트리

- > ML 알고리즘 중 직관적으로 이해하기 쉬운 알고리즘
- > 데이터에 있는 규칙을 학습을 통해 자동으로 찾아내 트리(Tree) 기반의 분류 규칙을 만든다.
- > if/else 기반으로 데이터를 분류하여 답을 찾아낸다.
- > 최대한 균일한 데이터 세트를 구성할 수 있도록 분할하는 것이 필요하다.
 - 균일한 데이터 : 정보를 쉽게 예측할 수 있는 데이터
- > 정보 이득과 지니 계수를 이용하여 균일한 데이터인지 측정 가능하다.

• 결정 트리

- > 규칙 노드 : 규칙 조건이 되는 노드로 if/else로 데이터를 구분한다.
- > 리프 노드 : 결정된 클래스 값이다.
- > 서브 트리 : 새로운 규칙 조건마다 생성된 구조

056



• 결정 트리

> 정보 이득

- 엔트로피라는 개념을 기반으로 한다.
- 엔트로피에서 전체 자식 노드의 엔트로피를 뺀 값으로 정의.
- 결정 트리는 정보 이득 지수가 최대값이 되는 분기를 찾는다.

$$H(B) = -\sum_{i=1}^2 p_i(y|B) \log p_i(y|B) \quad \text{IG}(A, \{B, C\}) = H(A) - H(\{B, C\}) = H(A) - \frac{|B|}{|B|+|C|} H(B) - \frac{|C|}{|B|+|C|} H(C)$$

> 지니 계수

- 클래스가 잘못 분류될 확률의 가중 평균으로 정의한다.
- 0이 가장 평등하고 1로 갈수록 불평등하다.
- 결정 트리는 지니 계수가 최소값이 되는 분기를 찾는다.

$$G_A = \sum_{i=1}^2 p_i(1-p_i) = \sum_{i=1}^2 (p_i - p_i^2) = 1 - \sum_{i=1}^2 p_i^2 \quad \text{Gini_impurity}(k, t_k) = \frac{|B|}{|B|+|C|} G_B + \frac{|C|}{|B|+|C|} G_C$$

• 결정 트리

058

> 결정 트리 분할 구조

1. 데이터 집합의 모든 아이템이 같은 분류에 속하는지 확인
- 2-1. 리프 노드로 만들어서 분류 결정
- 2-2. 데이터를 분할하는 데 가장 좋은 속성과 분할 기준을 찾음
(정보 이득 or 지니 계수를 이용한다)
3. 해당 속성과 분할 기준으로 데이터 분할하여 Branch 노드 생성
4. 처음으로 돌아가 모든 데이터 집합의 분류가 결정될 때까지 수행

> 결정 트리의 장단점

- 쉽고 직관적이라는 장점
- 데이터 균일도만 신경쓰면 피쳐의 스케일링이나 정규화 같은 전처리의 영향도가 크지 않다.
- 단점으로는 과적합으로 인해 정확도가 떨어질 수 있다.

• 결정 트리

> 결정 트리 파라미터

- max_depth
 - 깊이의 상한선
- min_samples_split
 - 노드에서 분기를 진행하는 최소한의 샘플 숫자
- min_samples_leaf
 - 리프 노드에 있을 샘플 개수의 최솟값(과적합을 막고자 사용)
- max_features
 - 각 노드에서 분기를 확인할 피쳐 수
- max_leaf_nodes
 - 말단 노드(Leaf)의 최대 개수
- criterion
 - 분기 규칙 선택('gini' : 지니불순도 최소화, 'entropy' : 정보이득 최대화)

• 결정 트리 실습

> 와인 데이터 실습

```
from sklearn.datasets import load_wine
import pandas as pd
```

```
wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)
df['class'] = wine.target
print(wine.target_names)
df.head()
```

Out : ['class_0' 'class_1' 'class_2']

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04

- 결정 트리 실습

- > 학습 데이터 준비

```
from sklearn.model_selection import train_test_split

X_data = df.iloc[:, :-1]
y_data = df.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data,
                                                    test_size=0.2 , random_state= 156)

print(X_train.shape, X_test.shape)
print(y_train.shape, y_test.shape)
```

Out : (142, 13) (36, 13)
(142,) (36,)

- 결정 트리 실습

> 학습 / 예측 / 평가

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

dt_clf = DecisionTreeClassifier(random_state=156)
dt_clf.fit(X_train , y_train)
pred = dt_clf.predict(X_test)
accuracy = accuracy_score(y_test , pred)
print('결정 트리 예측 정확도: {0:.4f}'.format(accuracy))
```

Out : 결정 트리 예측 정확도: 0.9444

- 결정 트리 실습

- > GridSearchCV

```
from sklearn.model_selection import GridSearchCV

parameters = {'criterion':['gini', 'entropy'],
              'max_depth':[None, 2, 3, 5, 7],
              'min_samples_split':[2,3,5,7],
              'min_samples_leaf':[1,3,5,7]}

grid_dt = GridSearchCV(dt_clf, param_grid=parameters, cv=5)
grid_dt.fit(X_train, y_train)

scores_df = pd.DataFrame(grid_dt.cv_results_)
scores_df[['params', 'mean_test_score', 'rank_test_score']]
```

- 결정 트리 실습

> GridSearchCV

Out :

	params	mean_test_score	rank_test_score
0	{'criterion': 'gini', 'max_depth': None, 'min_...	0.851478	101
1	{'criterion': 'gini', 'max_depth': None, 'min_...	0.851478	101
2	{'criterion': 'gini', 'max_depth': None, 'min_...	0.858621	68
3	{'criterion': 'gini', 'max_depth': None, 'min_...	0.865517	65
4	{'criterion': 'gini', 'max_depth': None, 'min_...	0.858621	68
...	...	...	...
155	{'criterion': 'entropy', 'max_depth': 7, 'min_...	0.858621	68
156	{'criterion': 'entropy', 'max_depth': 7, 'min_...	0.865764	33
157	{'criterion': 'entropy', 'max_depth': 7, 'min_...	0.865764	33
158	{'criterion': 'entropy', 'max_depth': 7, 'min_...	0.865764	33
159	{'criterion': 'entropy', 'max_depth': 7, 'min_...	0.865764	33

- 결정 트리 실습

065

- > GridSearchCV

```
print('GridSearchCV 최적 파라미터:', grid_dt.best_params_)  
print(f'GridSearchCV 최고 정확도: {grid_dt.best_score_:.4f}')
```

Out : GridSearchCV 최적 파라미터: {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5}
GridSearchCV 최고 정확도: 0.8936

```
model = grid_dt.best_estimator_  
pred = model.predict(X_test)  
accuracy_score(y_test, pred)
```

Out : 1.0

• 다중 클래스 평가 지표

- > 지금까지 배운 정밀도/재현율/F1은 이진 분류 기준이었다. 클래스가 셋 이상인 와인 데이터 실습에서는 어떻게 평가할까?
- > 다중 클래스에서는 각 클래스를 한 번씩 Positive로 두고 나머지를 Negative로 보는 방식으로 정밀도/재현율을 클래스별로 각각 계산한다.
- > `sklearn.metrics.classification_report`를 사용하면 클래스별 정밀도/재현율/F1을 한 번에 확인할 수 있다.
 - 매크로 평균(macro avg) : 클래스별 지표를 단순 평균 (클래스 크기 무시)
 - 가중 평균(weighted avg) : 클래스별 데이터 개수(support)로 가중 평균
 - 마이크로 평균(micro avg) : 모든 클래스의 TP·FP·FN을 합친 후 지표 계산

- 결정 트리 실습

067

> classification_report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, pred,
                           target_names=wine.target_names))
```

Out :

	precision	recall	f1-score	support
class_0	0.92	1.00	0.96	11
class_1	1.00	0.86	0.92	14
class_2	0.92	1.00	0.96	11
accuracy			0.94	36
macro avg	0.94	0.95	0.95	36
weighted avg	0.95	0.94	0.94	36

- 결정 트리 실습

- > 피쳐 중요도 그래프

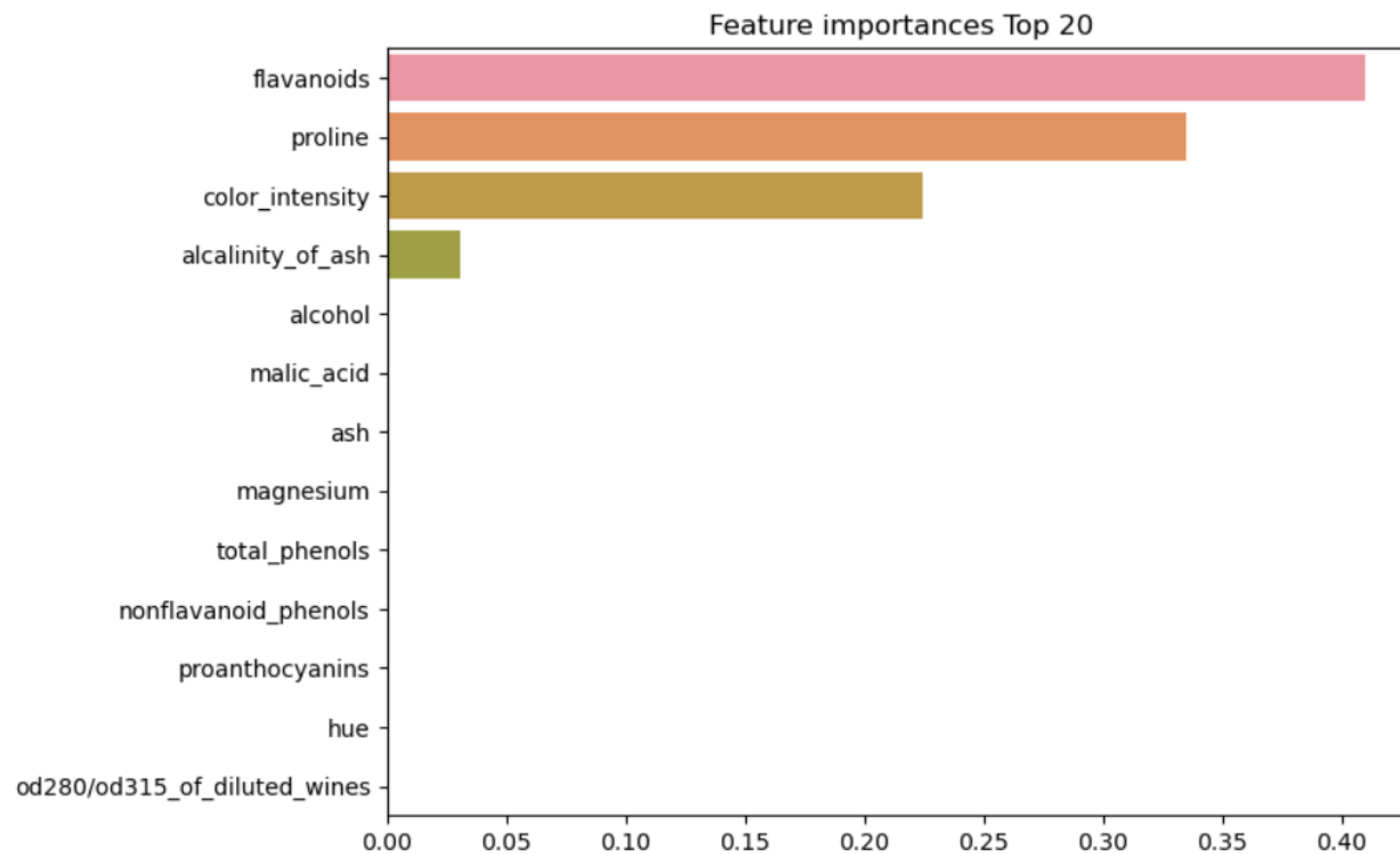
```
import seaborn as sns
import matplotlib.pyplot as plt

ftr_importances_values = model.feature_importances_
ftr_importances = pd.Series(ftr_importances_values, index=X_train.columns)
ftr_top20 = ftr_importances.sort_values(ascending=False)[:20]
plt.figure(figsize=(8,6))
plt.title('Feature importances Top 20')
sns.barplot(x=ftr_top20 , y = ftr_top20.index)
plt.show()
```

- 결정 트리 실습

- > 피쳐 중요도 그래프

Out :



- 결정 트리 실습

- > 트리 구조 그래프

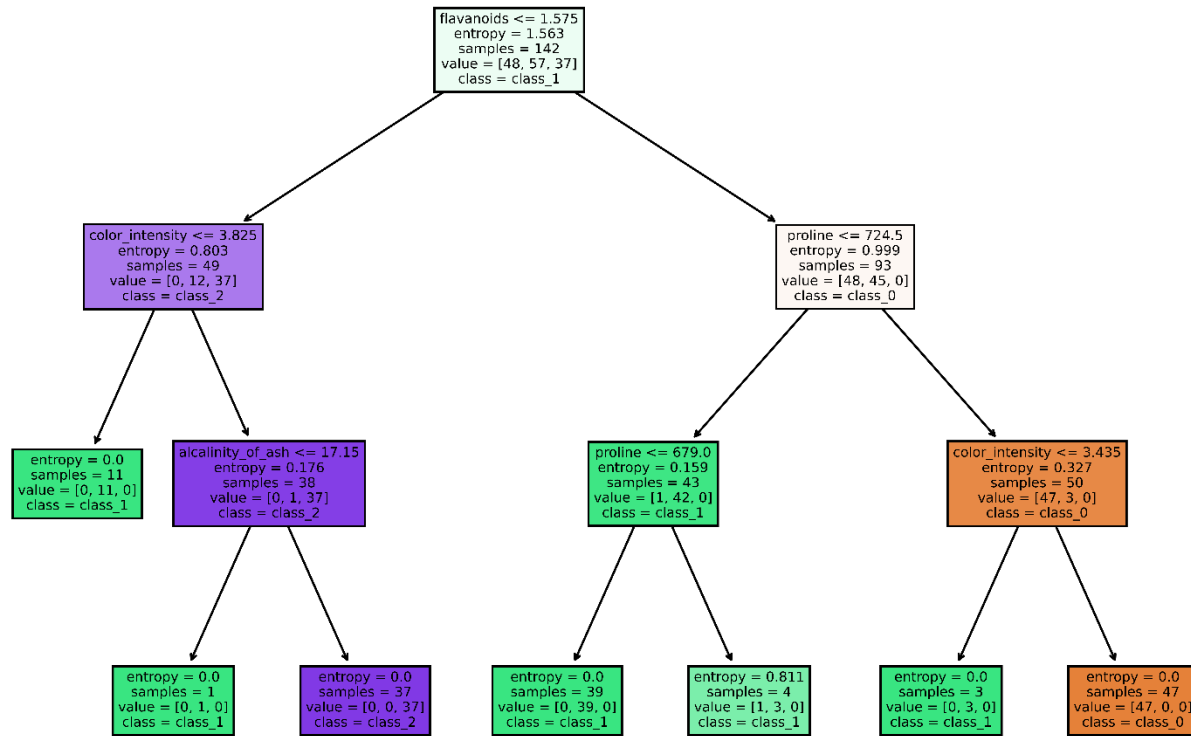
```
from sklearn.tree import plot_tree

plt.figure(figsize=(10,7), dpi=1200)
plot_tree(model, filled=True,
          feature_names=wine.feature_names,
          class_names=list(wine.target_names))
```

• 결정 트리 실습

> 트리 구조 그래프

Out :



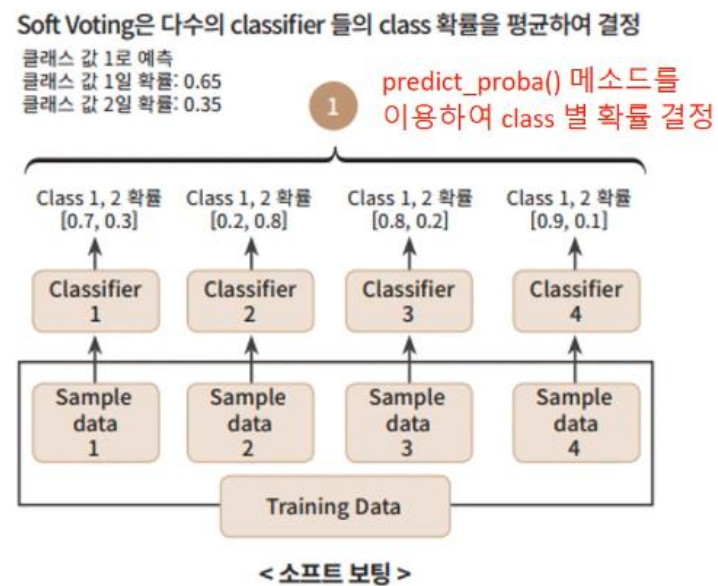
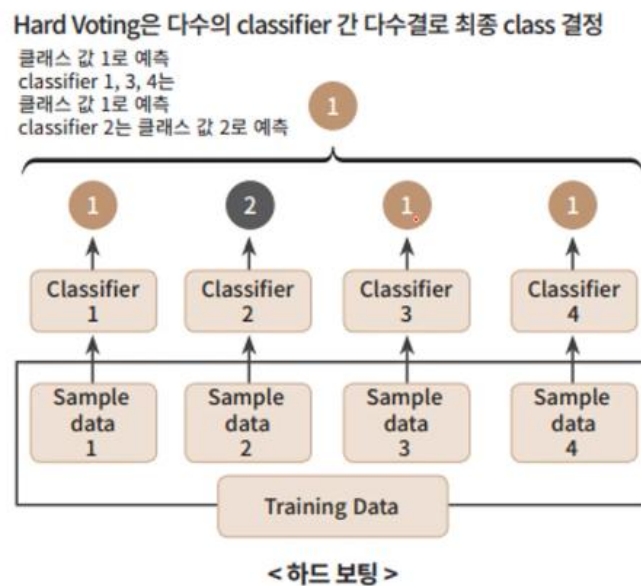
• 앙상블 학습

- > 여러 개의 분류기(Classifier)를 생성하고 그 예측을 결합함으로써 보다 정확한 예측을 도출하는 법
- > 앙상블 기법의 종류
 - 보팅(Voting) : 한 데이터셋에서 다른 알고리즘을 사용
 - 배깅(Bagging) : 한 분류기로 서로 다른 데이터 샘플을 학습
 - 부스팅(Boosting) : 순차적으로 학습-예측하면서 오류를 개선해가며 가중치를 주는 방식
 - 스택킹(stack) : 여러 모델을 통해 나온 결과를 학습하여 예측하는 방식

• 앙상블 학습

> 보팅 유형

- 하드 보팅
- 소프트 보팅



- 앙상블 학습

- > 보팅 분류기(Voting Classifier)

```
import pandas as pd
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cancer = load_breast_cancer()
data_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
data_df.head(3)
```

• 앙상블 학습

> 보팅 분류기(Voting Classifier)

Out :

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...
0	17.99	10.38	122.8	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...
1	20.57	17.77	132.9	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...
2	19.69	21.25	130.0	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...
	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	
	25.38	17.33	184.6	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890	
	24.99	23.41	158.8	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902	
	23.57	25.53	152.5	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758	

- 앙상블 학습

- > 보팅 분류기(Voting Classifier)

```
lr_clf = LogisticRegression(solver='liblinear')
dt_clf = DecisionTreeClassifier(random_state=0)
vo_clf = VotingClassifier( estimators=[('LR',lr_clf),
                                       ('DT',dt_clf)] , voting='soft' )

X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
                                                    test_size=0.2 , random_state= 0 )

vo_clf.fit(X_train , y_train)
pred = vo_clf.predict(X_test)
print(f'Voting 분류기 정확도: {accuracy_score(y_test,pred):.4f}')
```

Out : Voting 분류기 정확도: 0.9123

- 앙상블 학습

- > 보팅 분류기(Voting Classifier)

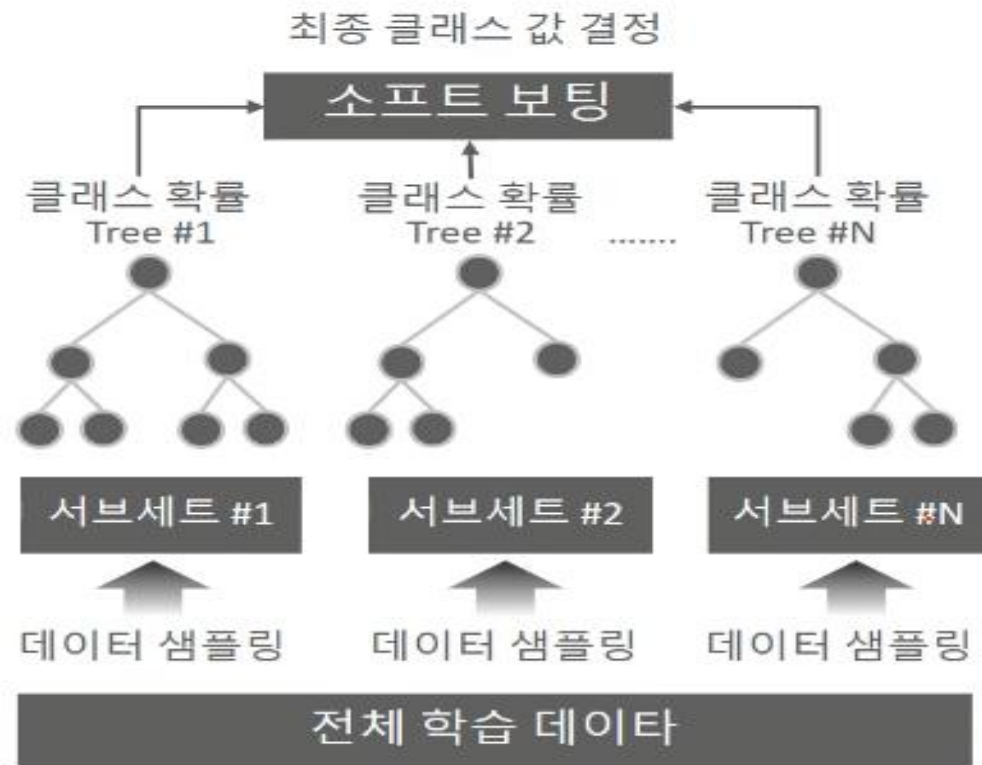
```
# 각 모델의 정확도 계산
classifiers = [lr_clf, dt_clf]
for clf in classifiers:
    clf.fit(X_train , y_train)
    pred = clf.predict(X_test)
    class_name= clf.__class__.__name__
    print(f'{class_name} 정확도: {accuracy_score(y_test, pred):.4f}')
```

Out : LogisticRegression 정확도: 0.9561
DecisionTreeClassifier 정확도: 0.9123

• 앙상블 학습

> 랜덤 포레스트

- 배깅의 대표적인 알고리즘
- 결정 트리 기반의 앙상블 학습



• 앙상블 학습

> 부트스트래핑

- 원본 데이터에서 복원추출하여 여러 학습 표본을 만드는 기술



- 앙상블 학습

- > 랜덤포레스트 실습

```
from sklearn.ensemble import RandomForestClassifier

cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
                                                    test_size=0.2 , random_state= 0)

rf_clf = RandomForestClassifier(random_state=0)
rf_clf.fit(X_train , y_train)
pred = rf_clf.predict(X_test)
accuracy = accuracy_score(y_test , pred)
print(f'랜덤 포레스트 정확도: {accuracy:.4f}')
```

Out : 랜덤 포레스트 정확도: 0.9649

- 앙상블 학습

- > 랜덤포레스트 하이퍼 파라미터 및 튜닝

```
from sklearn.model_selection import GridSearchCV

params = {'n_estimators':[100],
          'max_depth' : [6, 8, 10, 12],
          'min_samples_leaf' : [8, 12, 18 ],
          'min_samples_split' : [8, 16, 20]}

rf_clf = RandomForestClassifier(random_state=0)
grid_cv = GridSearchCV(rf_clf , param_grid=params , cv=3, n_jobs=-1, verbose=True)
grid_cv.fit(X_train , y_train)
```

- 앙상블 학습

082

- > 랜덤포레스트 하이퍼 파라미터 및 튜닝

```
print('GridSearchCV 최적 파라미터:', grid_cv.best_params_)  
print(f'GridSearchCV 최고 정확도: {grid_cv.best_score_:.4f}')
```

Out : GridSearchCV 최적 파라미터: {'max_depth': 6, 'min_samples_leaf': 8, 'min_samples_split': 8, 'n_estimators': 100}
GridSearchCV 최고 정확도: 0.9385

```
model = grid_cv.best_estimator_  
pred = model.predict(X_test)  
accuracy_score(y_test, pred)
```

Out : 0.9649122807017544

- 앙상블 학습

- > 피쳐 중요도 그래프

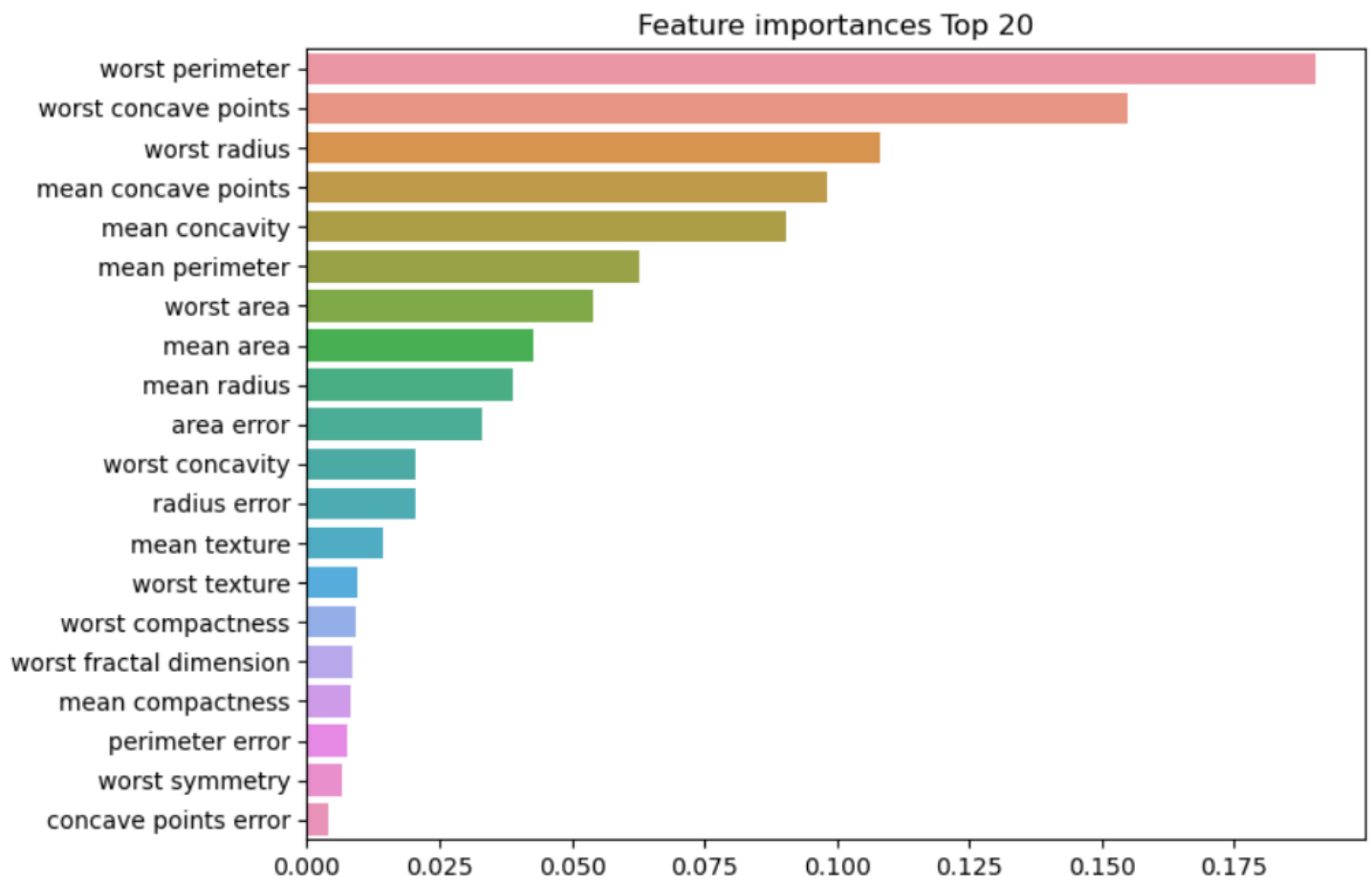
```
import matplotlib.pyplot as plt
import seaborn as sns

ftr_importances_values = model.feature_importances_
ftr_importances = pd.Series(ftr_importances_values, index=cancer.feature_names )
ftr_top20 = ftr_importances.sort_values(ascending=False)[:20]
plt.figure(figsize=(8,6))
plt.title('Feature importances Top 20')
sns.barplot(x=ftr_top20 , y = ftr_top20.index)
plt.show()
plt.draw()
```

• 앙상블 학습

> 피쳐 중요도 그래프

Out :



• 앙상블 학습

> GBM(Gradient Boosting Machine)

- 경사 하강법(Gradient Descent)를 사용하여 가중치 업데이트를 하는 방식

> Gradient Descent

- 오류 값: 실제 값 - 예측 값
- 예측 함수를 $F(x)$ 라고 하면 오류식 $h(x)=y-F(x)$
- 이 오류식 $h(x)=y-F(x)$ 를 최소화 하는 방향성을 가지고
- 반복적으로 가중치 값을 업데이트 하는 것이 Gradient Descent

> GBM 하이퍼 파라미터

- loss : 경사 하강법에서 사용할 비용 함수 지정
- learning_rate : 학습을 진행할 때마다 적용하는 학습률, 0.1
- n_estimators : weak learner의 개수
- subsample : weak learner가 학습에 사용하는 데이터 샘플링의 비율
- 나머지 의사결정나무의 파라미터와 같다.

- 앙상블 학습

- > GBM(Gradient Boosting Machine)

```
from sklearn.ensemble import GradientBoostingClassifier

cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
                                                    test_size=0.2 , random_state= 0)

gb_clf = GradientBoostingClassifier(random_state=0)
gb_clf.fit(X_train , y_train)
gb_pred = gb_clf.predict(X_test)
gb_accuracy = accuracy_score(y_test, gb_pred)
print(f'GBM 정확도: {gb_accuracy:.4f}')
```

Out : GBM 정확도: 0.9649

- XGBoost(eXtra Gradient Boost)

- > 트리 기반 앙상블 학습에서 각광받는 알고리즘 중 하나

- 병렬 CPU 환경에서 병렬 학습이 가능

- > 주요 장점

- 예측 성능이 뛰어남
- GBM 대비 빠른 수행 시간
- Regularization(과적합 규제)
- Tree pruning(나무 가지치기) : 이득이 없는 분할을 가지치기
- 자체 내장 교차 검증
- 결손값 자체 처리

- XGBoost(eXtra Gradient Boost)

- > 주요 파라미터

- eta : learning rate와 같은 파라미터
- num_boost_round : weak learner의 개수
- min_child_weight : 분할 조절
- gamma : 리프 노드를 나눌지 결정하는 최소 값
- max_depth : 트리의 깊이
- objective : 손실함수 정의, (binary:logistic, multi:softmax)
- eval_metric : 검증 유형

- XGBoost(eXtra Gradient Boost)

- > XGBoost 실습

```
import xgboost as xgb
from xgboost import plot_importance
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')
```

- XGBoost(eXtra Gradient Boost)

> XGBoost 실습

[illegible]

- XGBoost(eXtra Gradient Boost)

- > XGBoost 실습

```
dtr = xgb.DMatrix(data=X_tr, label=y_tr)
dval = xgb.DMatrix(data=X_val, label=y_val)
dtest = xgb.DMatrix(data=X_test , label=y_test)

params = {'max_depth':3,'eta': 0.05,'objective':'binary:logistic','eval_metric':'logloss'}
num_rounds = 400
```

- XGBoost(eXtra Gradient Boost)

092

> XGBoost 실습

```
eval_list = [(dtr,'train'),(dval,'eval')]
```

```
# 또는 eval_list = [(dval,'eval')]
```

```
xgb_model = xgb.train(params = params , dtrain=dtr ,  
                      num_boost_round=num_rounds ,  
                      early_stopping_rounds=50, evals=eval_list )
```

```
Out: [0]      train-logloss:0.65016    eval-logloss:0.66183  
      [1]      train-logloss:0.61131    eval-logloss:0.63609  
      [2]      train-logloss:0.57563    eval-logloss:0.61144  
      [3]      train-logloss:0.54310    eval-logloss:0.59204
```

- XGBoost(eXtra Gradient Boost)

093

> XGBoost 실습

```
pred_probs = xgb_model.predict(dtest)
print('predict( ) 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨')
print(np.round(pred_probs[:10],3))
```

```
preds = [ 1 if x > 0.5 else 0 for x in pred_probs ]
print('예측값 10개만 표시:',preds[:10])
```

Out : predict() 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨
[0.938 0.004 0.75 0.049 0.98 1. 0.999 0.999 0.998 0.001]
예측값 10개만 표시: [1, 0, 1, 0, 1, 1, 1, 1, 1, 0]

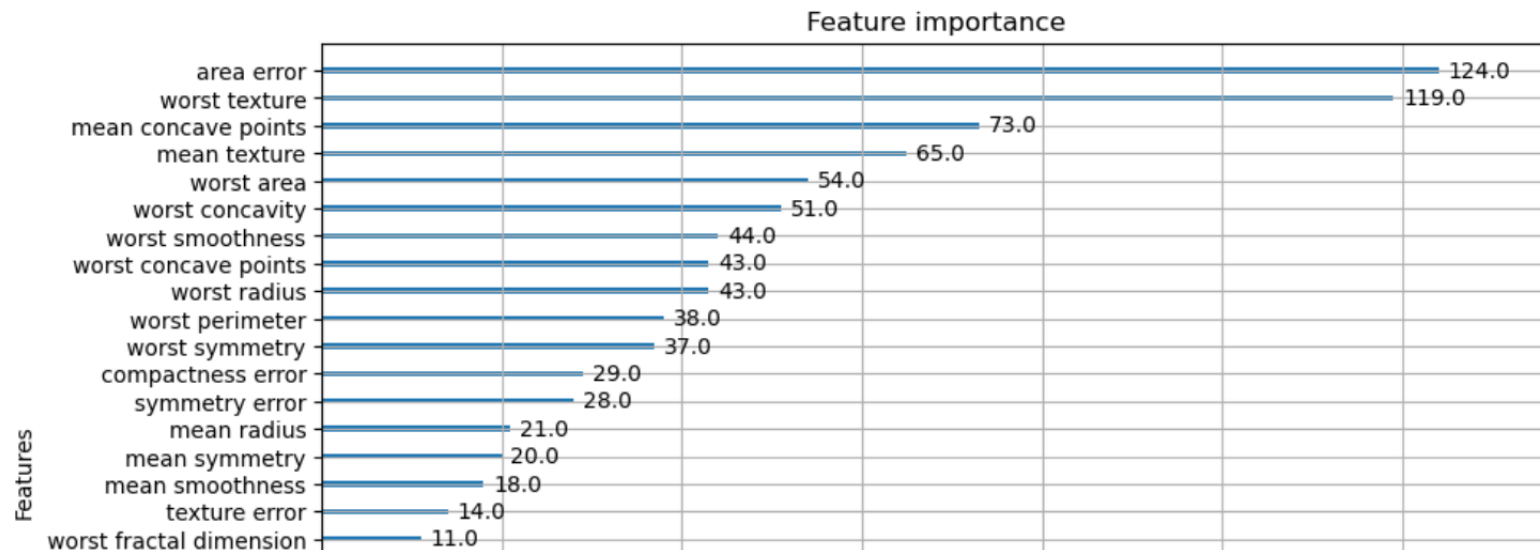
- XGBoost(eXtra Gradient Boost)

094

- > XGBoost 실습

```
import matplotlib.pyplot as plt  
fig, ax = plt.subplots(figsize=(10, 7))  
plot_importance(xgb_model, ax=ax)
```

Out :



- XGBoost(eXtra Gradient Boost)

095

> 사이킷런 래퍼 XGBoost의 개요 및 적용

```
from xgboost import XGBClassifier

xgb_wrapper = XGBClassifier(n_estimators=400, learning_rate=0.1, max_depth=3)
evals = [(X_test, y_test)]
xgb_wrapper.fit(X_train, y_train, early_stopping_rounds = 100,
                eval_metric = "logloss", eval_set=evals)
preds = xgb_wrapper.predict(X_test)
pred_proba = xgb_wrapper.predict_proba(X_test)[:, 1]
```

- XGBoost(eXtra Gradient Boost)

> XGBoost 실습

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score
from sklearn.metrics import recall_score, f1_score, roc_auc_score
confusion = confusion_matrix( y_test, pred)
accuracy = accuracy_score(y_test , pred)
precision = precision_score(y_test , pred)
recall = recall_score(y_test , pred)
f1 = f1_score(y_test,pred)
roc_auc = roc_auc_score(y_test, pred_proba)
print('오차 행렬')
print(confusion)
print(f'정확도: {accuracy:.4f}, 정밀도: {precision:.4f}, 재현율: {recall:.4f}')
print(f'F1: {f1:.4f}, AUC:{roc_auc:.4f}')
```

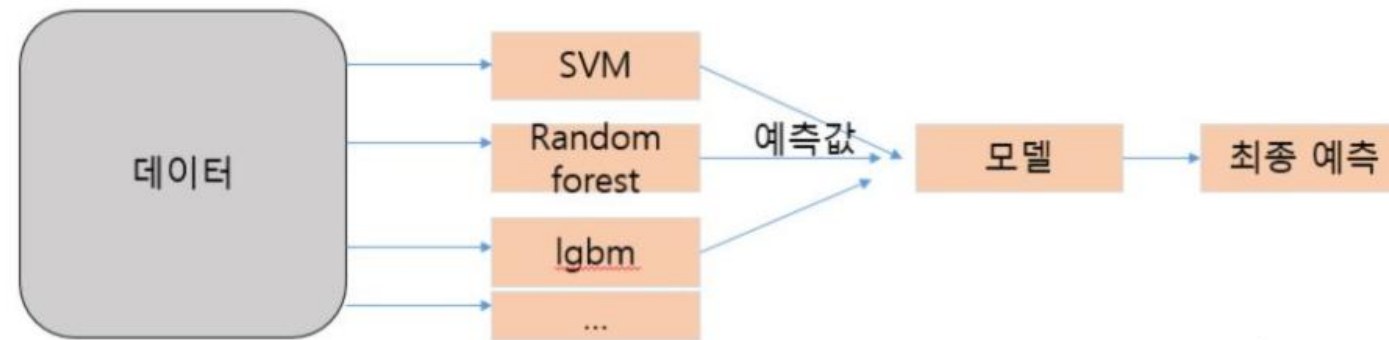
- XGBoost(eXtra Gradient Boost)

> XGBoost 실습

Out : 오차 행렬
[[34 3]
[1 76]]
정확도: 0.9649, 정밀도: 0.9620, 재현율: 0.9870
F1: 0.9744, AUC:0.9951

- 스택킹 앙상블

- > 기본구조



- > 성능이 비슷한 여러 모델로 예측한 값을 최종 모델에 적용하여 예측을 수행

- 스택킹 앙상블

- > 기본 스택킹 모델

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

# 4개의 모델(KNN, RF, Ada, DT)를 통해 개별 예측을 하고
# 최종으로 Logistic 모델을 사용하여 예측 예정
```

- 스택킹 앙상블

- > 기본 스택킹 모델

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cancer_data = load_breast_cancer()
X_data = cancer_data.data
y_label = cancer_data.target
X_train , X_test , y_train , y_test = train_test_split(X_data, y_label,
                                                         test_size=0.2 , random_state=0)
```

- 스택킹 앙상블

- > 기본 스택킹 모델

```
# 개별 ML 모델을 위한 Classifier 생성.  
knn_clf = KNeighborsClassifier(n_neighbors=4)  
rf_clf = RandomForestClassifier(n_estimators=100, random_state=0)  
dt_clf = DecisionTreeClassifier(random_state=0)  
ada_clf = AdaBoostClassifier(n_estimators=100)  
  
# 최종 Stacking 모델을 위한 Classifier 생성.  
lr_final = LogisticRegression(solver='liblinear')
```

- 스택킹 앙상블

- > 스택킹 실습

```
from sklearn.ensemble import StackingClassifier

stack = StackingClassifier([('knn', knn_clf),
                             ('rf', rf_clf),
                             ('dt', dt_clf),
                             ('ada', ada_clf)],
                           final_estimator = lr_final)
```

- 스택킹 앙상블

- > 스택킹 실습

```
stack.fit(X_train, y_train)
```

```
pred = stack.predict(X_test)
```

```
accuracy_score(y_test, pred)
```

Out : 0.9649122807017544

- 로지스틱 회귀

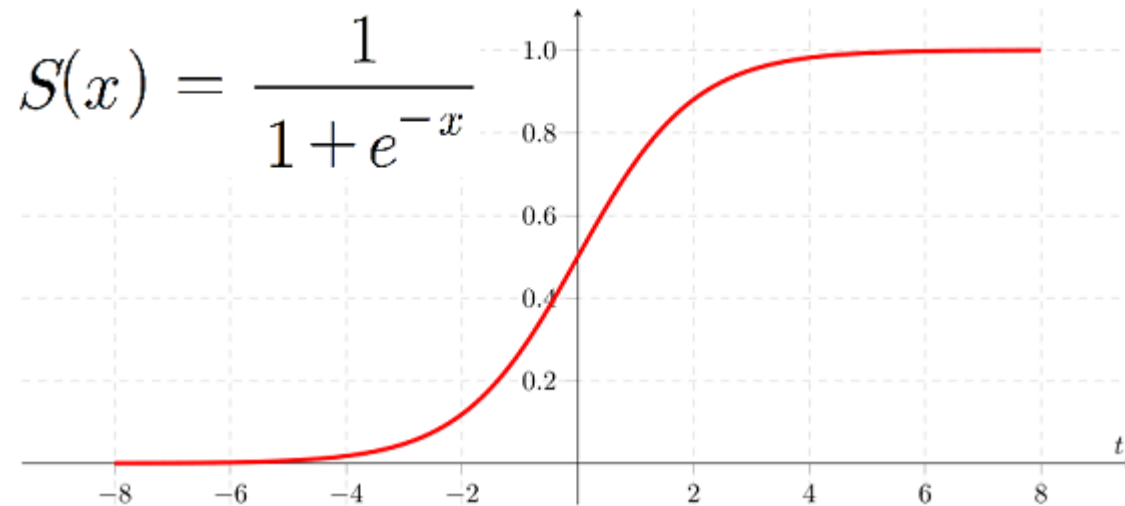
0104

- > 분류 문제(classification task)를 해결하는 가장 기본적인 머신러닝 모델
- > 로지스틱 회귀라는 이름에도 불구하고 회귀가 아닌 분류에 사용
- > 로지스틱 회귀 모델은 주어진 이진 목표값 y 의 클래스 레이블 값이 1이 나올 확률 $p(y)$ 와 피쳐 벡터 x 사이의 관계를 모델링하는 기법

$$\log \frac{p(y_i | x_i, w)}{1 - p(y_i | x_i, w)} = w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p} = \sum_{j=0}^p w_j x_{i,j}$$

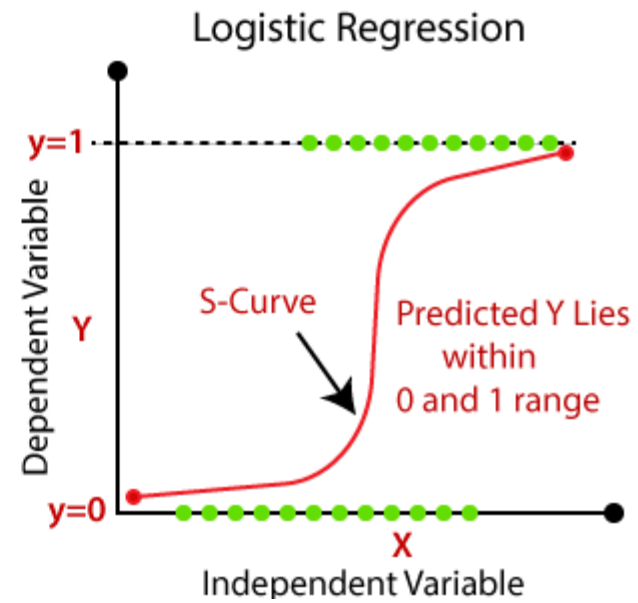
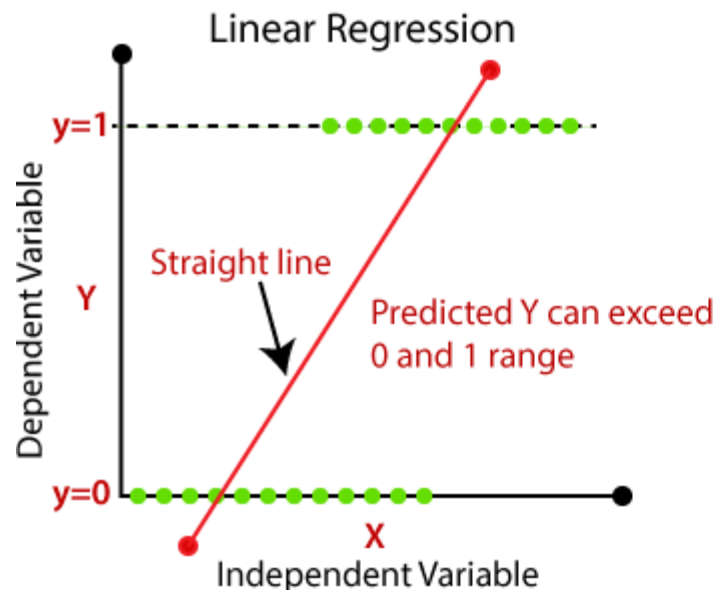
- 로지스틱 회귀

- > 선형 회귀 방식을 분류에 적용한 알고리즘
- > 아래와 같은 시그모이드 함수를 생성하고
- > 이를 토대로 입력 특성의 선형 결합을 시그모이드 함수에 넣어 양성 확률을 추정하고, 임계값으로 클래스를 결정한다



- 로지스틱 회귀

- > 선형 회귀와의 비교



- 로지스틱 회귀

- > 로지스틱 회귀 실습

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score
import warnings
warnings.filterwarnings('ignore')

cancer = load_breast_cancer()
scaler = StandardScaler()
data_scaled = scaler.fit_transform(cancer.data)
```

- 로지스틱 회귀

0108

- > 로지스틱 회귀 실습

```
X_train , X_test, y_train , y_test = train_test_split(data_scaled, cancer.target,
                                                    test_size=0.3, random_state=0)

lr_clf = LogisticRegression()
lr_clf.fit(X_train, y_train)
pred = lr_clf.predict(X_test)
pred_proba = lr_clf.predict_proba(X_test)[:,-1]
acc = accuracy_score(y_test, pred)
auc = roc_auc_score(y_test, pred_proba)

print(f'accuracy: {acc:.3f}, roc_auc:{auc:.3f}')
```

Out : accuracy: 0.977, roc_auc:0.972

- 로지스틱 회귀

- > 로지스틱 회귀 실습

```
solvers = ['lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga']

for solver in solvers:
    lr_clf = LogisticRegression(solver=solver, max_iter=600)
    lr_clf.fit(X_train, y_train)
    pred = lr_clf.predict(X_test)
    pred_proba = lr_clf.predict_proba(X_test)[:,1]
    acc = accuracy_score(y_test, pred)
    auc = roc_auc_score(y_test, pred_proba)

    print(f'solver: {solver}, accuracy: {acc:.3f}, roc_auc:{auc:.3f}')
```

- 로지스틱 회귀

0110

- > 로지스틱 회귀 실습

Out : solver: lbfgs, accuracy: 0.977, roc_auc:0.995
solver: liblinear, accuracy: 0.982, roc_auc:0.995
solver: newton-cg, accuracy: 0.977, roc_auc:0.995
solver: sag, accuracy: 0.982, roc_auc:0.995
solver: saga, accuracy: 0.982, roc_auc:0.995

하이퍼파라미터	의미	특징
'newton-cg'	켈레기울기법 conjugate gradient method	희소 데이터셋 sparse dataset 에서 추천한다.
'lbfgs'	L-BFGS-B 알고리즘	작은 크기의 데이터셋에 추천하며, 데이터의 크기가 크 다면 성능이 떨어질 수 있다.
'liblinear'	최적화된 좌표 하강법 coordinate descent 알고리즘	좌표 하강법은 4장에서 자세히 다루도록 한다. 데이터 의 크기가 작을 때 추천한다.
'sag'	SAG(stochastic average gradient descent) 알고리즘	샘플 개수와 피쳐 개수가 모두 큰 대형 데이터셋에서 빠 르게 수렴한다. 하지만 빠른 수렴은 각 피쳐의 스케일 이 비슷할 때만 보장되므로 <code>sklearn.preprocessing</code> 모듈의 클래스를 사용해 데이터를 전처리하는 것이 좋다.
'saga'	SAG 알고리즘의 변형	'sag'와 마찬가지로 대형 데이터셋에서 잘 동작하고, 피쳐 스케일에 대한 정규화를 선행하는 것이 좋다.

- 로지스틱 회귀

- > 로지스틱 회귀 실습

```
from sklearn.model_selection import GridSearchCV

params={'solver':['liblinear', 'lbfgs'],
        'penalty':['l2', 'l1'],
        'C':[0.01, 0.1, 1, 5, 10]}

lr_clf = LogisticRegression()
grid_clf = GridSearchCV(lr_clf, param_grid=params, scoring='accuracy', cv=3 )
grid_clf.fit(data_scaled, cancer.target)

print(f'최적 하이퍼 파라미터:{grid_clf.best_params_}')
print(f'최대 평균 정확도:{grid_clf.best_score_:.3f}')
```

• 로지스틱 회귀

> 로지스틱 회귀 실습

Out : 최적 하이퍼 파라미터: {'C': 0.1, 'penalty': 'l2', 'solver': 'liblinear' }

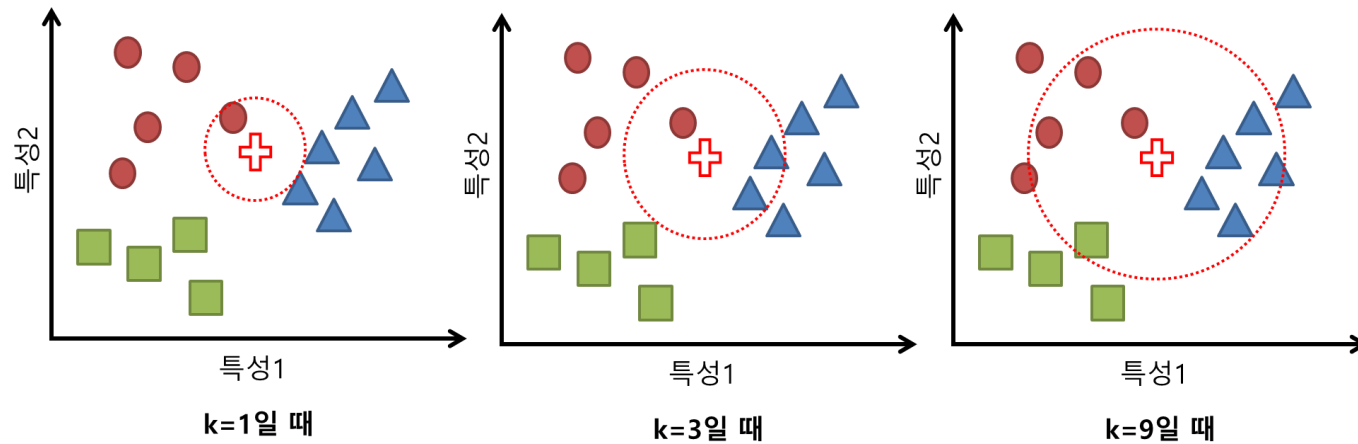
최대 평균 정확도: 0.979

하이퍼파라미터	주요값	기본값	의미
penalty	'l1', 'l2', 'elasticnet', 'none'	'l2'	규제 페널티 선택 - 'l1': L1 규제 적용 - 'l2': L2 규제 적용 - 'elasticnet': 엘라스틱 넷 규제 적용 - 'none': 규제 미부여
C	float>0	1.0	규제 페널티 크기의 역수

기능	'liblinear'	'lbfgs'	'newton-cg'	'sag'	'saga'
다항 분류 + L2 규제	X	O	O	O	O
OVR + L2 규제	O	O	O	O	O
다항 분류 + L1 규제	X	X	X	X	O
OVR + L1 규제	O	X	X	X	O
엘라스틱 넷 규제	X	X	X	X	O
규제 미부여 기능	X	O	O	O	O

- 최근접 이웃(K-Neighbors)

- > 데이터의 산포도를 기준으로 가까운 데이터를 같은 분류로 판단하여 학습하고 예측한다.
- > 데이터의 분포만으로 결정을 하기에 간단하고 빠르게 분류한다.
- > k라는 파라미터를 조절하면 결과가 달라진다.



- 최근접 이웃(K-Neighbors)

- > 최근접 이웃 실습

```
from sklearn.datasets import load_breast_cancer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score

cancer = load_breast_cancer()
scaler = StandardScaler()
data_scaled = scaler.fit_transform(cancer.data)

X_train , X_test, y_train , y_test = train_test_split(data_scaled, cancer.target,
                                                         test_size=0.3, random_state=0)
```

- 최근접 이웃(K-Neighbors)

- > 최근접 이웃 실습

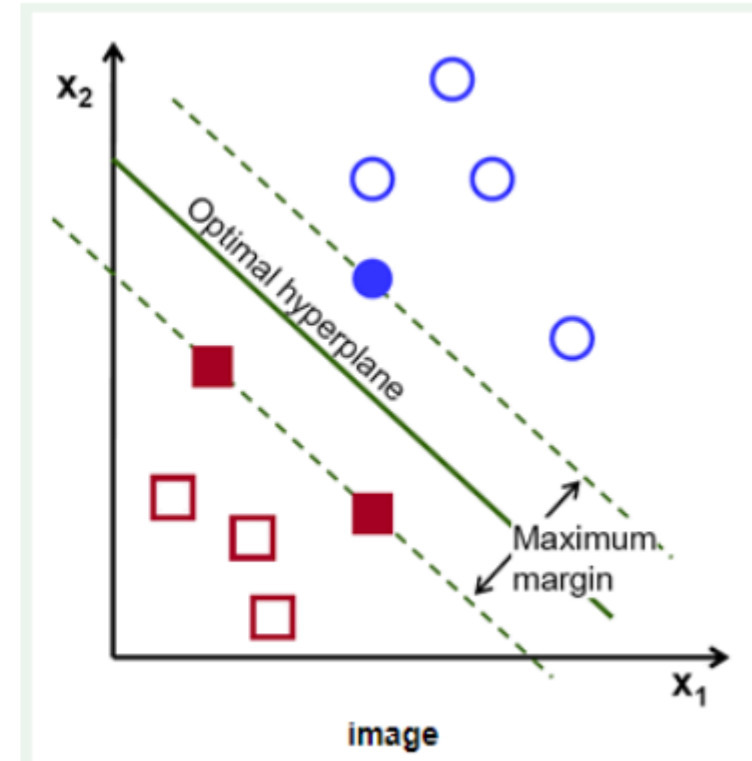
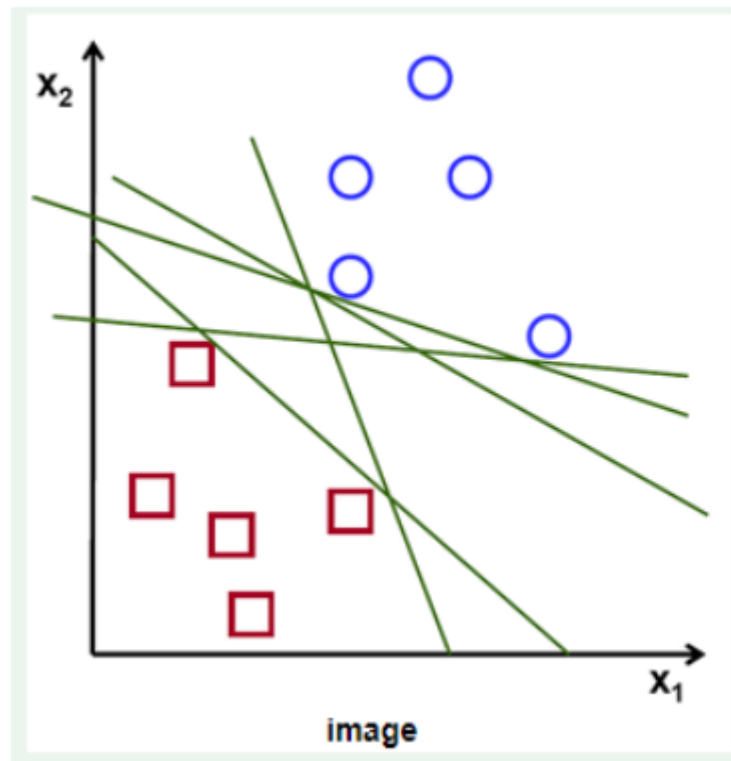
```
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)
pred_proba = knn.predict_proba(X_test)[:,:1]
acc = accuracy_score(y_test, pred)
auc = roc_auc_score(y_test, pred_proba)

print(f'accuracy: {acc:.3f}, roc_auc:{auc:.3f}')
```

Out : accuracy: 0.953, roc_auc:0.992

- 서포트 벡터 머신(SVM)

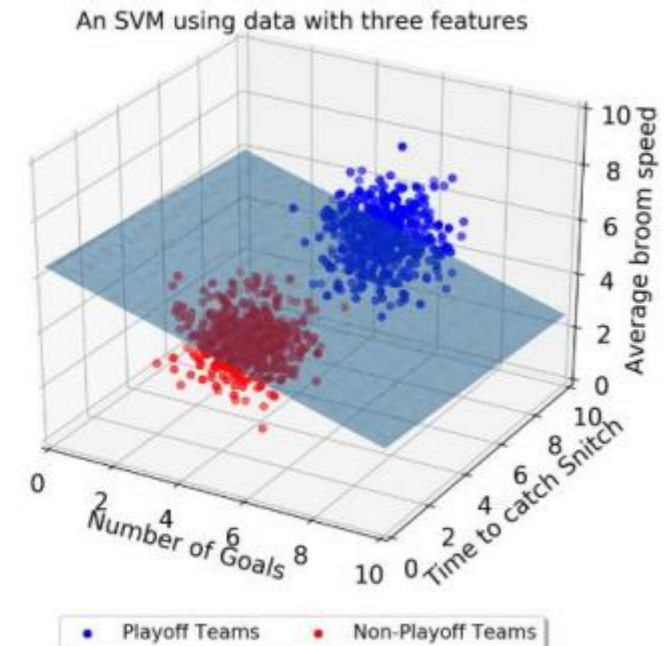
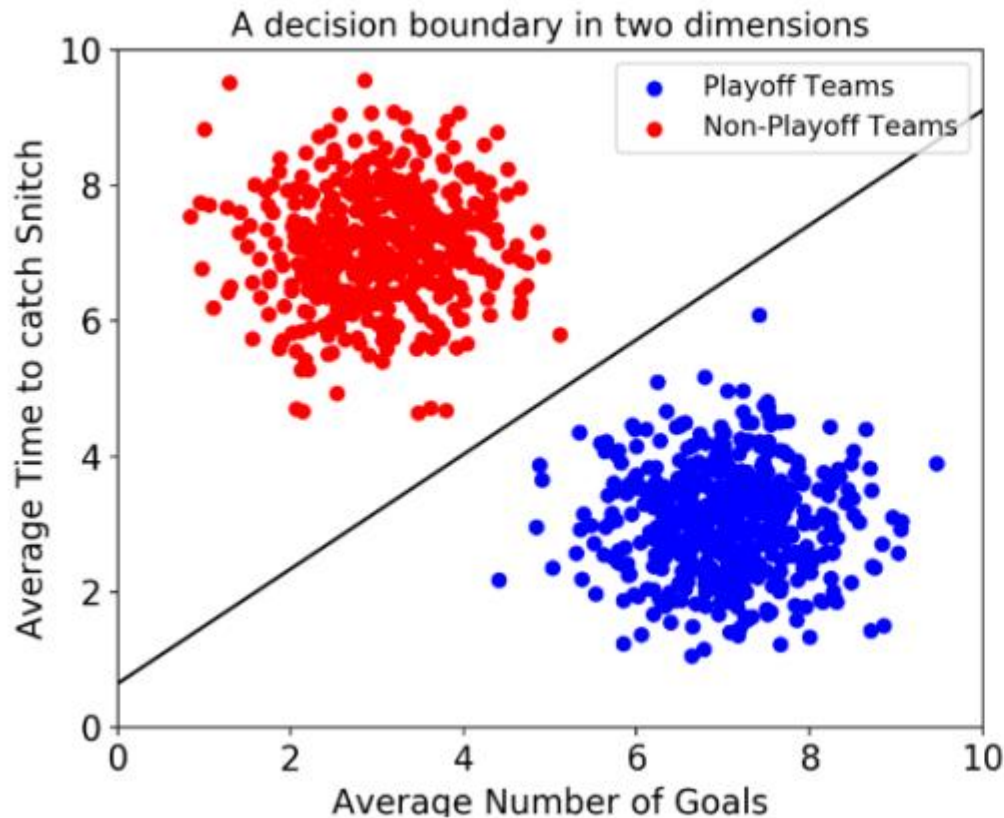
- > SVM은 기본적으로 분류를 위한 기준 선을 정의하여 두 개의 데이터를 분리하는 방법으로 데이터들과 거리가 가장 먼(Margin 극대화) 초평면을 선택하여 분리하는 방법이다.



- 서포트 벡터 머신(SVM)

- > 일반적으로 선으로 보일 순 있으나 평면을 만들어 분리한다.
- > 이를 인지할 수 있는 범위는 3차원 까지이다. (최대 독립변수 3개, 종속변수 1개)

0117



- 서포트 벡터 머신(SVM)

- > SVM 실습

```
from sklearn.datasets import load_breast_cancer
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score

cancer = load_breast_cancer()
scaler = StandardScaler()
data_scaled = scaler.fit_transform(cancer.data)

X_train , X_test, y_train , y_test = train_test_split(data_scaled, cancer.target,
                                                         test_size=0.3, random_state=0)
```

- 서포트 벡터 머신(SVM)

- > SVM 실습

```
svc = SVC(probability = True)
svc.fit(X_train, y_train)
pred = svc.predict(X_test)
pred_proba = svc.predict_proba(X_test)[:,1]
acc = accuracy_score(y_test, pred)
auc = roc_auc_score(y_test, pred_proba)

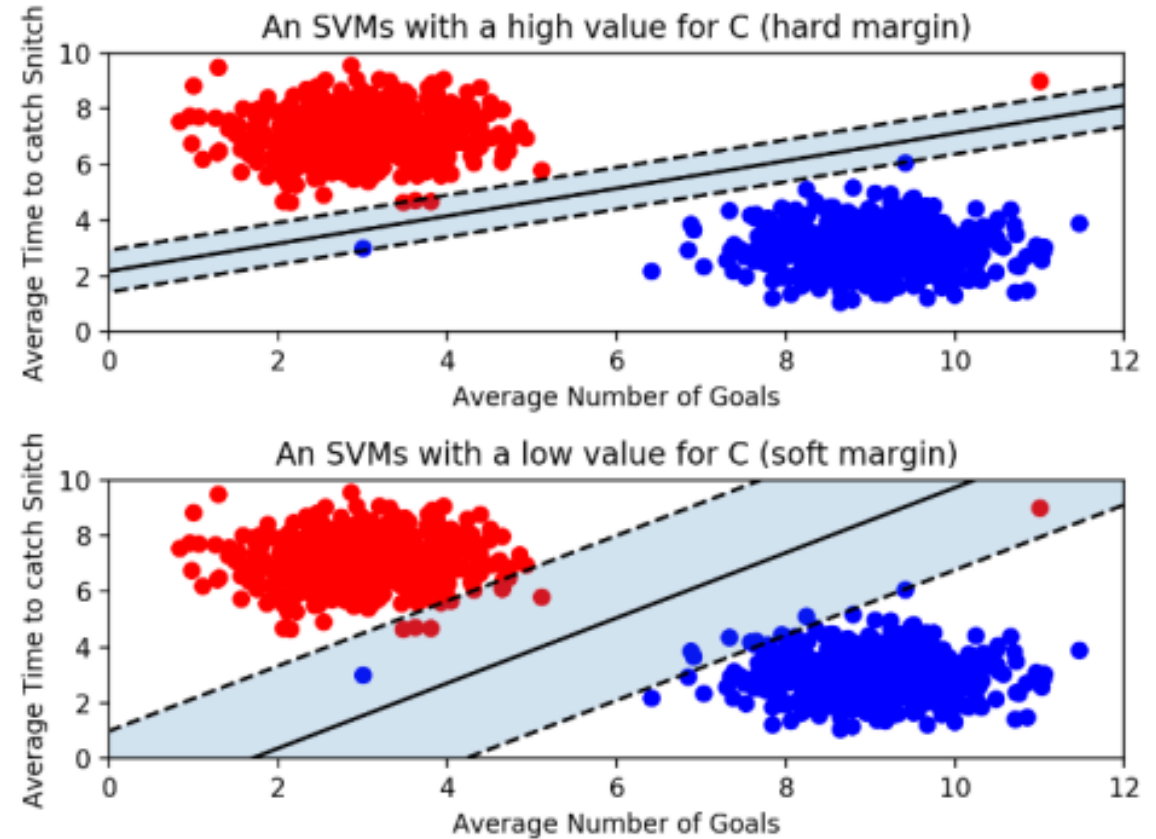
print(f'accuracy: {acc:.3f}, roc_auc:{auc:.3f}')
```

Out : accuracy: 0.977, roc_auc:0.998

• 서포트 벡터 머신(SVM)

> 특징

- 두 집단을 잘 분리하기 위해
경계점(Suport Vector)를 만들어
구분한다.
- 오른쪽 그림처럼 이상치가 존재하면
오류를 최소화하기 위해 마진줄인다.
- 이를 해결하기 위해 하드 마진이 아닌
소프트 마진 방법을 이용한다.
- 다른 모델에 비해 이상치에 민감하고
스케일링 유무의 차이가 심하다.

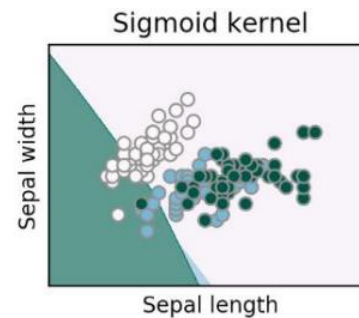
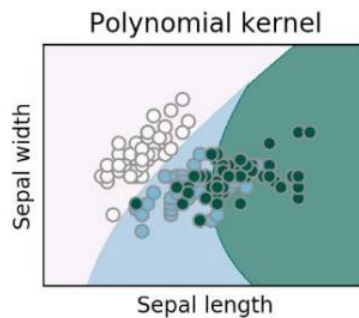
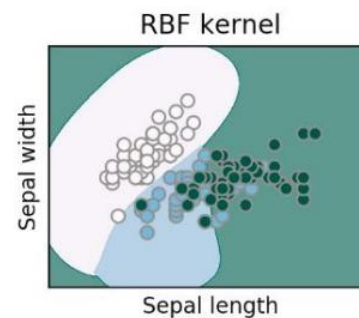
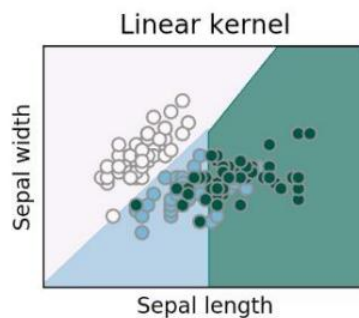


• 서포트 벡터 머신(SVM)

> 커널 종류

- 사용하는 커널에 따라 초평면의 모양이 달라지고 하이퍼 파라미터가 조금 차이난다.

커널 명칭	형태	비고
선형 커널linear kernel	$K(x_1, x_2) = x_1^T x_2$	커널을 정의하지 않는 선형 SVM과 동일하다.
다항 커널polynomial kernel	$K(x_1, x_2) = (\gamma x_1^T x_2 + r)^d$	계수coefficient에 해당하는 γ , 절편intercept에 해당하는 r , 차수degree에 해당하는 d 를 가진다. 선형 커널을 확장한 개념으로, $\gamma=1, r=0, d=1$ 일 때 선형 커널과 같다.
RBF 커널radial basis function kernel	$K(x_1, x_2) = e^{-\gamma \ x_1 - x_2\ ^2}$	분산의 역수에 해당하는 계수 γ 를 가진다. 가우시안 분포Gaussian distribution와 비슷한 형태이므로 가우시안 커널Gaussian kernel이라고도 한다. SVM 모델에서 가장 많이 사용하는 커널 함수의 하나이다.
시그모이드 커널sigmoid kernel	$K(x_1, x_2) = \tanh(\gamma x_1^T x_2 + r)$	γ 는 함수의 기울기를 조정하는 계수이며, r 은 절편에 해당한다.



• 서포트 벡터 머신(SVM)

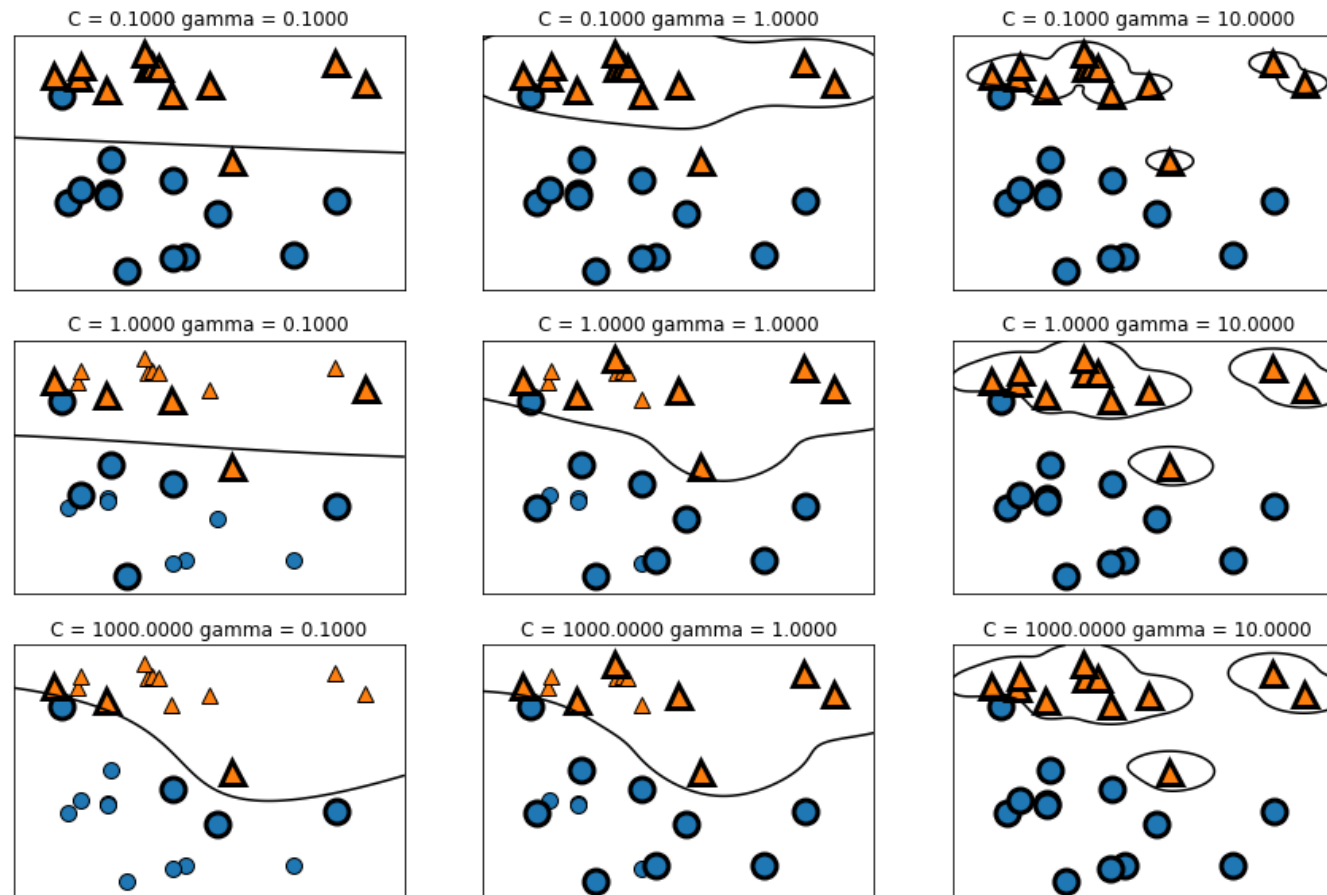
> 하이퍼 파라미터 종류

- C : 값이 클수록 마진을 줄여 오차를 줄이고 작을수록 마진을 늘린다.
- Gamma : 근처에 있는 것에 가중치 정도 (커널 계수)
- degree : kernel이 'poly'일 때 차수
- Coef0 : kernel이 'poly'나 'sigmoid'일 때 커널 함수의 절편

하이퍼파라미터	주요값	기본값	의미
C	float>0	1.0	규제 강도의 역수
kernel	'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'	'rbf'	사용할 커널의 종류. 사용자 지정 함수를 사용할 수 있다.
probability	bool	False	확률적 추정을 사용할지의 여부. 교차검증법(cross validation)을 사용하므로 속도가 떨어질 수 있다.
tol	float>0	1e-3	정지 조건에 대한 허용 오차
class_weight	None, 'balanced', dict	None	클래스 가중치 설정. 클래스 불균형(class imbalance) 효과를 줄이고자 할 때 유용하다. - None: 모든 클래스에 같은 가중치 부여 - 'balanced': 클래스 빈도에 반비례하는 가중치 설정 - dict: 사용자 지정 가중치 부여
max_iter	int>0 또는 int=-1	-1	이테레이션의 상한선 - max_iter=-1: 상한선 없음. tol에 의한 정지 조건에 도달할 때까지 계속 학습한다.
decision_function_shape	'ovr', 'ovo'	'ovr'	다중 클래스일 때 반환할 결정 함수의 종류 설정. 이진 분류라면 무시한다. - 'ovr': OVR(one-vs-rest) 결정 함수 - 'ovo': libsvm의 OVO(one-vs-one) 결정 함수
random_state	None, int	None	probability=True에 한해 데이터 셔플링(data shuffling)에 대한 랜덤성을 제어

- 서포트 벡터 머신(SVM)

> 하이퍼 파라미터 종류에 따른 변화



- 나이브 베이즈(Naive Bayes)

- > 베이즈 정리(Bayes' Theorem)를 기반으로 조건부 확률을 계산하여 분류하는 알고리즘.
- > 모든 피처가 서로 독립적(나이브, Naive)이라고 가정하고 각 피처가 클래스에 속할 확률을 곱해 계산한다.
- > 학습 데이터가 적어도 비교적 안정적으로 동작하며 연산 속도가 빠르다.
- > 스팸 메일 분류, 텍스트(뉴스 기사 등) 분류 등에 널리 사용된다.
 - 가우시안 나이브 베이즈(GaussianNB) : 연속형 데이터에 사용
 - 다항 나이브 베이즈(MultinomialNB) : 텍스트(단어 빈도) 데이터에 주로 사용
- > 단점 : 피처 간 독립 가정이 실제 데이터에서는 잘 성립하지 않는 경우가 많다.